UNIVERSITÉ DU LAC TANGANYIKA

Faculté d'Informatique - Département de Génie Logiciel



Université du Lac Tanganyika

Conception d'un Système de Gestion des Stocks d'Engrais (Distribution) : Cas de la Société FOMI au Burundi

Mémoire de Fin d'Études en Génie Logiciel

Présenté par :

Ishimwe Antosper Davy

Itangiteka Dickson Polland

Sous la direction de :

Prof. Mme Providence

Année Académique 2025-2026

DÉDICACE

Nous dédions ce travail à nos chers parents, pour leur soutien indéfectible tout au long de notre parcours académique, ainsi qu'à toutes nos familles et à nos amis qui nous ont encouragé dans cette aventure intellectuelle.

Nous le dédions également à tous les agriculteurs burundais qui, au prix d'efforts considérables, contribuent quotidiennement à la sécurité alimentaire de notre nation. Que ce modeste travail puisse, à son échelle, contribuer à améliorer les conditions de travail et de vie de ceux qui nourrissent notre pays.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à Madame la directrice de notre mémoire pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de la réalisation de ce travail. Sa rigueur et sa patience ont été des qualités essentielles qui ont permis l'aboutissement de ce projet.

Nos remerciements s'adressent également aux membres du corps enseignant du Département de Génie Logiciel de la Faculté d'informatique de l'Université du Lac Tanganyika pour la qualité de la formation reçue au cours de ces années d'études. Les connaissances acquises en programmation, en bases de données, en génie logiciel et en architecture logicielle ont constitué le socle sur lequel repose ce travail.

Nous remercions le Directeur Commercial de FOMI — Fertilisants Organo-Minéraux Industries pour son accueil chaleureux, sa collaboration qui nous a été essentielle à la réalisation de ce projet. Les échanges avec le Directeur ont permis de comprendre en profondeur les réalités opérationnelles qui motivent ce projet.

Nous remercions également les membres du jury qui accepteront d'évaluer ce travail pour leurs remarques constructives qui contribueront à son amélioration.

Enfin, Nous remercions nos familles et nos amis pour leur soutien moral et leur patience tout au long de ces années d'études. Leur présence bienveillante a été une source d'inspiration et de motivation constante.

RÉSUMÉ

Le Burundi, pays d'Afrique de l'Est à vocation essentiellement agricole, fait face à des défis majeurs en matière de gestion des intrants agricoles, notamment les engrais. La société FOMI — Fertilisants Organo-Minéraux Industries, premier producteur d'engrais organo-minéraux au Burundi, joue un rôle central dans la chaîne d'approvisionnement via le Programme National de Subvention des Engrais (PNSEB). Cependant, le manque de digitalisation de ce système d'approvisionnement engendre des ruptures de stock chroniques, des pertes d'informations, des risques de fraude et des retards dans la distribution aux agriculteurs.

Ce mémoire présente la conception et la réalisation d'un Système dédiée à la gestion centralisée des stocks d'engrais de la société FOMI. L'application, développée en Java 21 avec le framework Spring Boot 3.3.5 et la base de données PostgreSQL 16, implémente un système complet couvrant l'authentification par JWT avec contrôle d'accès basé sur les rôles, la gestion FIFO des lots par date, un système d'alertes automatiques de rupture et de stock bas, un workflow de validation des tickets de distribution, ainsi qu'un audit trail complet de toutes les opérations.

Le projet compte 218 tests automatisés (215 tests d'intégration et 3 tests unitaires), tous validés. La solution répond aux 25 besoins fonctionnels identifiés et intègre des mécanismes de sécurité avancés. Ce travail constitue une contribution significative à la modernisation de la gestion des stocks agricoles au Burundi et ouvre des perspectives d'évolution vers une plateforme numérique complète intégrant les clients web et mobile déjà développés, les notifications SMS et les paiements mobiles.

TABLE DES MATIÈRES

INTRODUCTION GÉNÉRALE 8
1. Contexte et justification 8
1.1 Le contexte géographique et économique du Burundi 8
1.2 La fertilité des sols et l'usage des engrais 8
1.3 La réforme administrative de 2023 9
2. Problématique 10
2.1 Problèmes identifiés et impacts 10
2.2 Conséquences pour les parties prenantes 12
3. Objectifs 12
3.1 Objectif global 12
3.2 Objectifs spécifiques 12
4. Hypothèses de recherche 13
5. Méthodologie de recherche 13
6. Intérêt du sujet 14
7. Délimitation du sujet 14
8. Articulations 14
CHAPITRE I : PRÉSENTATION DU LIEU DE STAGE 16
I.1 Introduction 16
I.2 Historique de l'institution 16
I.3 Situation géographique 16
I.4 Vision, missions et objectifs de l'institution 17
I.5 Structure organisationnelle 18
I.6 Déroulement du stage 18

I.6.1 Introduction 18
I.6.2 Activités réalisées 19
I.6.3 Étude de l'existant 19
I.6.3.1 Description de l'existant 19
I.6.3.2 Critique de l'existant 22
I.6.3.3 Solution proposée 24
I.6.4 Corrélation du stage avec les Éléments Constitutifs des Unités d'Enseignement 24
CHAPITRE II : CONCEPTS THÉORIQUES ET MÉTHODOLOGIE DE L'ÉTUDE 26
II.1 Concepts théoriques 26
II.1.1 Gestion des stocks et méthode FIFO 26
II.1.2 Traçabilité et piste d'audit 26
II.1.3 Technologies et outils utilisés 27
II.1.4 État de l'art : systèmes de référence en Afrique de l'Est 29
II.2 Méthodologie de l'étude 31
II.2.1 Démarche méthodologique 31
II.2.2 Phases du projet 31
II.2.3 Outils de développement et de validation 31
II.2.4 Méthode de validation des résultats 32
CHAPITRE III : PRÉSENTATION DES RÉSULTATS 33
III.1 Analyse des besoins 33
III.1.1 Besoins fonctionnels 33
III.1.2 Règles métier 35
III.1.3 Cas d'utilisation 36
III.1.4 Besoins non fonctionnels 39
III.2 Conception du système 41

III.2.1 L'architecture du système 41
III.2.2 Modèle de données 43
III.2.3 Diagramme de séquence 49
III.3 Réalisation, tests et évaluation 50
III.3.1 Structure du projet 51
III.3.2 Système d'alertes 52
III.3.3 Workflow de validation des tickets 53
III.3.4 Annulation des mouvements 53
III.3.5 Statistiques et tableaux de bord 54
III.3.6 Piste d'audit 55
III.3.7 Tests et déploiement 56
III.3.8 Évaluation des résultats 58
CONCLUSION GÉNÉRALE 63
CRITIQUES, SUGGESTIONS ET RECOMMANDATIONS 64
1. Critiques 64
2. Suggestions 65
3. Recommandations 65
RÉFÉRENCES BIBLIOGRAPHIQUES 66
WEBOGRAPHIE 67
Annexes 68
Annexe A : Extraits de code 68
Annexe B : Liste des endpoints API 71
Annexe C : Schéma de la base de données (extrait migration SQL) 75
Annexe D : Résultats des tests 78

INTRODUCTION GÉNÉRALE

1. Contexte et justification

Le Burundi, petite nation enclavée de la région des Grands Lacs en Afrique de l'Est, présente un profil économique et démographique singulier. Malgré des ressources naturelles limitées et une superficie modeste, le pays tire l'essentiel de son revenu national de l'agriculture, secteur qui emploie la grande majorité de sa population active. Comprendre le contexte dans lequel évolue la société FOMI est essentiel pour appréhender les enjeux de la gestion des stocks d'engrais au Burundi. Ce chapitre pose le cadre géographique, économique et agronomique qui justifie la pertinence et l'urgence d'un système d'information pour la gestion des intrants agricoles.

1.1 Le contexte géographique et économique du Burundi

La faiblesse des rendements s'explique par plusieurs facteurs convergents qui se renforcent mutuellement. L'érosion des sols, aggravée par la culture sur pente et les pluies torrentielles de la saison humide, emporte chaque année des milliers de tonnes de terre arable. Le manque d'intrants agricoles de qualité — semences améliorées, engrais, produits phytosanitaires — prive les cultures des nutriments essentiels à leur croissance. L'absence quasi-totale de mécanisation agricole limite la superficie cultivable par ménage et réduit l'efficacité des travaux champêtres. Enfin, les pratiques culturales traditionnelles, bien qu'adaptées au contexte local, ne permettent pas d'optimiser les rendements sans apport extérieur d'intrants.

Dans ce contexte, l'accès à des engrais de qualité à prix abordable revêt une importance capitale pour améliorer la productivité agricole et, par conséquent, la sécurité alimentaire du pays. Les études agronomiques montrent qu'un apport correct en engrais peut multiplier les rendements par deux à trois dans les sols, transformation qui pourrait à elle seule éliminer les déficits alimentaires chroniques du pays.

1.2 La fertilité des sols et l’usage des engrais

La fertilité des sols constitue l'un des défis les plus pressants de l'agriculture burundaise. Les sols du Burundi sont majoritairement acides. Cette acidité est particulièrement problématique

car elle réduit les éléments nutritifs pourtant essentiel au développement racinaire et à la formation des graines.

L'utilisation d'engrais chimiques au Burundi reste très en deçà des besoins réels. La consommation totale oscillait historiquement entre 15 000 et 25 000 tonnes par an, soit une consommation par hectare de l'ordre de 39,79 kg/ha en 2022. Cette consommation est très inférieure à la recommandation technique de 100 à 200 kg/ha selon les cultures et les types de sol. Pour comparaison, le Kenya consomme environ 40 kg/ha, la Tanzanie 15 kg/ha, et les pays d'Europe de l'Ouest entre 150 et 300 kg/ha. Le Burundi se situe donc dans une zone de sous-fertilisation chronique qui compromet directement la productivité agricole.

L'acidité des sols aggrave considérablement le problème de la fertilisation. Sans apport correcteur de chaux ou d'amendements calcaires, l'efficacité des engrais minéraux est réduite de 30 à 50 %. Les sols acides fixent le phosphore sous des formes insolubles (aluminium-phosphate, fer-phosphate), rendant cet élément nutritif inaccessible aux racines des plantes. De plus, les sols fortement acides (pH < 5,0) libèrent de l'aluminium et du manganese en quantités toxiques pour les racines, entravant leur développement et réduisant leur capacité d'absorption des nutriments. Cette situation appelle des solutions intégrées combinant amendements et fertilisants, ce qui justifie le positionnement stratégique des engrais organo-minéraux produits par FOMI, qui contiennent naturellement des amendements calciques.

1.3 La réforme administrative de 2023

Bien que votée par le législateur à travers la loi organique n'1/05 du 16 mars 2023, c'est en juillet 2025 que le Burundi a concrétisé la plus importante réforme de son organisation administrative depuis son indépendance en 1962. Le nombre de provinces a été drastiquement réduit de 18 à 5, et celui des communes est passé de 119 à 42. Les cinq nouvelles super-provinces institutionnelles sont désormais **Buhumuza**, **Bujumbura**, **Burunga**, **Butanyerera** et **Gitega**. Cette réorganisation, motivée par des impératifs de rationalisation budgétaire, de décentralisation et d'optimisation des services publics, a eu des répercussions directes et immédiates sur la chaîne nationale de distribution des intrants agricoles subventionnés.

La reconfiguration territoriale a entraîné la redistribution des zones de compétence, la restructuration des dépôts provinciaux (historiquement calqués sur les anciens Bureaux Provinciaux BPEAE) et la réorganisation complète des circuits logistiques. Les anciennes limites administratives ne correspondant plus aux nouvelles circonscriptions, il a été nécessaire de revoir l'ensemble du maillage territorial de distribution, incluant la réaffectation des stocks physiques, le redéploiement des équipes d'encadrement sur les collines et la mise à jour des bases de données d'acheminement. Cette phase de transition a généré des perturbations systémiques temporaires notables dans l'approvisionnement des ménages ruraux, matérialisées par des goulots d'étranglement logistiques et des retards de livraison dans plusieurs communes.

Cette réforme historique met en évidence la nécessité cruciale d'un système de gestion informatisé et dynamique, capable de s'adapter instantanément aux mutations des structures de l'État. Un système rigide, dont l'arbre géographique serait figé dans son code source, s'avérerait incapable de s'ajuster aux nouvelles réalités géopolitiques sans une refonte logicielle lourde, manuelle et coûteuse. Cette considération a directement guidé les choix d'architecture de l'API de gestion des stocks développée dans ce travail, notamment en ce qui concerne la modélisation polymorphe de la hiérarchie géographique. Grâce à des niveaux de dépôts configurables et des relations inter-dépôts hautement flexibles, le système conçu permet l'ajout, la modification et la suppression de paliers géographiques à la volée, sans aucune modification du code applicatif.

2. Problématique

2.1 Problèmes identifiés et impacts

L’analyse du fonctionnement actuel du système de gestion des stocks de FOMI a permis d’identifier onze problèmes majeurs, regroupés en quatre catégories : gestion des stocks, distribution, suivi et transparence, et gouvernance. Ces problèmes ont été recensés à travers les entretiens avec les responsables de FOMI, l’observation directe sur le terrain et l’analyse des documents de suivi existants (registres papier, tableurs Excel).

ID	Catégorie	Problème	Impact
P1	Gestion stocks	Absence de suivi en temps réel des stocks	Ruptures inattendues, réactions tardives
P2	Gestion stocks	Mauvaise estimation des besoins	Sous/sur-approvisionnement, gaspillage
P3	Distribution	Difficulté à identifier agriculteurs non servis	Inéquité dans la distribution
P4	Suivi	Absence d'historique centralisé des	Impossibilité d'audit

ID	Catégorie	Problème	Impact
		mouvements	
P5	Gouvernance	Risque de fraude et détournement	Pertes financières, perte confiance
P6	Suivi	Manque de statistiques fiables	Décisions non éclairées
P7	Distribution	Retards chroniques dans la distribution	Pertes de récoltes, , baisse des rendements
P8	Gestion stocks	Difficulté à localiser les stocks physiques disponibles	Flux logistiques et acheminements inefficaces
P9	Gouvernance	Absence de lien entre le ticket (bon) et le lot physique	Impossibilité de traçabilité
P10	Gestion stocks	Absence d'alerte préventive de rupture par zone	Gestion logistique purement réactive
P11	Transparence	Incapacité des agriculteurs à consulter l'état de la commande	Déplacements inutiles, opacité du système

Le problème **P1**, l'absence de suivi en temps réel des stocks, est sans doute le plus critique car il est à l'origine de la plupart des autres dysfonctionnements. Sans visibilité immédiate sur les niveaux de stock par magasin zonal et par type d'engrais, les instances décisionnelles sont contraintes d'arbitrer sur la base de données obsolètes ou incomplètes. Les registres manuels et les fichiers Excel décentralisés utilisés actuellement s'avèrent structurellement incapables de consolider une image actualisée des flux à l'échelle du pays.

Le problème **P5**, le risque de fraude et de détournement vers le marché noir, est le corollaire direct de ce déficit de traçabilité algorithmique (**P4**, **P9**). En l'absence d'un enregistrement numérique strict de chaque mouvement de stock et d'une réconciliation automatique entre les tickets d'acompte et les numéros de lots physiques, il est techniquement impossible de détecter à temps les anomalies de distribution sur les collines ou de documenter les infractions devant les autorités de contrôle.

Enfin, le problème **P11**, l'incapacité des petits producteurs à vérifier l'état d'avancement de leur commande, s'avère particulièrement préjudiciable. Face à l'asymétrie d'information, les agriculteurs sont contraints d'effectuer de longs déplacements à pied vers les **points de distribution collinaires** temporaires pour s'enquérir de la disponibilité de leurs intrants, subissant régulièrement de lourdes pertes de temps lorsque les livraisons physiques ont pris du retard

2.2 Conséquences pour les parties prenantes

Ces dysfonctionnements affectent directement les trois catégories d'acteurs du dispositif. Pour les agriculteurs, les ruptures de stock et les retards de distribution compromettent les calendriers culturaux et réduisent les rendements. Pour FOMI, l'absence de traçabilité engendre des pertes d'exploitation, des vols de lots et des difficultés de planification. Pour l'État et le PNSEB, le manque de visibilité en temps réel affaiblit le pilotage de la subvention et la confiance des bénéficiaires.

En définitive, la conjugaison de ces problèmes montre que le dispositif actuel ne permet ni un pilotage fiable des stocks, ni une traçabilité effective des distributions. C'est ce constat qui justifie la conception d'un système d'information centralisé, sécurisé et auditable pour la gestion des stocks d'engrais de FOMI.

3. Objectifs

3.1 Objectif global

L'objectif global de ce travail est de concevoir et de réaliser un système d'information centralisé pour la gestion des stocks d'engrais de la société FOMI, afin de garantir la disponibilité des intrants, la traçabilité des mouvements et la transparence de la distribution aux agriculteurs bénéficiaires du PNSEB.

3.2 Objectifs spécifiques

De manière spécifique, ce travail vise à :

1. Créer un système centralisé pour la gestion des stocks d'engrais de la société FOMI, offrant une interface standardisée pour toutes les opérations de gestion de stock (réception FOMI, transfert, distribution, ajustement). Ce système doit être documenté, testé et sécurisé, prêt à être consommé par un client web ou mobile.

2. Implémenter un mécanisme de suivi en temps réel des niveaux de stock par dépôt, par type d'engrais et par lot, permettant de visualiser instantanément la situation des stocks à tous les niveaux de la hiérarchie géographique. Ce suivi doit inclure les stocks totaux par dépôt, les stocks détaillés par lot avec dates de péremption, et les tendances d'évolution.

3. Mettre en place un système d'alertes automatiques de rupture de stock, de stock bas, de péremption de lots et de retards de distribution, permettant une gestion anticipée et proactive des situations critiques. Les alertes doivent être classées par niveau de sévérité et déclencher des actions correctives automatiques lorsque c'est possible.

4. Implémenter un audit trail complet enregistrant chaque opération sur le système, avec identification de l'utilisateur, horodatage, action réalisée et détails des données modifiées. L'audit trail doit être consultable, filtrable et exportable pour les besoins de contrôle et d'investigation.

4. Hypothèses de recherche

Pour guider notre démarche, nous avons formulé les hypothèses suivantes :

— H1 : L’absence de suivi centralisé et en temps réel des stocks constitue la principale cause des ruptures et des retards de distribution.

— H2 : Un système fondé sur des rôles et sur une piste d’audit complète réduit significativement les risques de fraude et de détournement des intrants.

— H3 : Un algorithme FIFO associé à une gestion rigoureuse des lots améliore la rotation des stocks et limite les pertes par péremption.

— H4 : Un workflow de validation des tickets garantit le respect des quotas et des droits des agriculteurs bénéficiaires.

— H5 : Des alertes automatiques permettent une gestion proactive des ruptures, des stocks bas et des retards.

— H6 : Une architecture API REST découplée facilite l’intégration de clients web et mobiles ainsi que l’évolution du système.

5. Méthodologie de recherche

La méthodologie adoptée combine une approche qualitative et une démarche d’ingénierie logicielle. La collecte des données s’est appuyée sur des entretiens semi-directifs avec les responsables de FOMI, sur l’observation directe du circuit de distribution et sur l’analyse

documentaire des registres, tableurs et tickets utilisés sur le terrain. La modélisation a été réalisée avec UML et le développement a suivi un cycle de vie incrémental : analyse des besoins, conception, implémentation, tests et déploiement. La méthodologie est détaillée au chapitre II.

6. Intérêt du sujet

Sur le plan scientifique, ce travail contribue à la réflexion sur la digitalisation de la chaîne d'approvisionnement en intrants agricoles dans les pays en développement, en proposant une architecture reproductible et documentée.

Sur le plan pratique, il dote FOMI d'un outil centralisé de suivi des stocks et des distributions, réduisant les ruptures, les pertes et les risques de fraude, et facilitant la prise de décision.

Sur le plan social, l'amélioration de la distribution des engrais contribue directement à la productivité agricole, à la sécurité alimentaire et au bien-être des ménages ruraux burundais.

Sur le plan personnel, ce stage nous a permis de mettre en pratique les connaissances acquises en programmation, bases de données, génie logiciel, architecture et sécurité informatique.

7. Délimitation du sujet

Délimitation spatiale : l'étude porte sur la société FOMI, son usine de Maramvya et son réseau de distribution (provinces, zones et collines) au Burundi.

Délimitation temporelle : les travaux ont été menés durant l'année académique 2025-2026, sur la base des données de la campagne agricole en cours.

Délimitation thématique : le travail couvre la gestion des stocks, les mouvements inter-dépôts, les tickets de distribution, les alertes et l'audit. Les aspects comptables, la paie, les ressources humaines et l'intégration des paiements mobiles ne sont pas traités.

8. Articulations

Outre l'introduction générale et la conclusion générale, ce rapport s'articule autour de trois chapitres. Le chapitre I présente le lieu de stage : historique, situation géographique, vision, missions, structure organisationnelle, déroulement du stage et étude de l'existant. Le chapitre

II expose les concepts théoriques mobilisés et la méthodologie de l'étude. Le chapitre III présente les résultats : analyse des besoins, conception, implémentation, tests, déploiement et évaluation. Le rapport se termine par les critiques, suggestions et recommandations, puis la bibliographie et les annexes.

CHAPITRE I : PRÉSENTATION DU LIEU DE STAGE

I.1 Introduction

Le présent chapitre présente le cadre institutionnel de notre stage de fin d'études effectué au sein de la société FOMI — Fertilisants Organo-Minéraux Industries. Il retrace l'historique de l'entreprise, sa situation géographique, sa vision, ses missions et ses objectifs, sa structure organisationnelle, avant de décrire le déroulement du stage, les activités réalisées, l'étude de l'existant et la corrélation entre les activités du stage et les éléments constitutifs des unités d'enseignement suivies au cours de notre formation.

I.2 Historique de l'institution

FOMI — Fertilisants Organo-Minéraux Industries — a été fondée en février 2019 par M. Adrien Ntigacika, à travers le groupe ITRACOM Holdings. Il s'agit de la première et unique usine de production d'engrais organo-minéraux au Burundi, marquant ainsi une étape historique dans l'autonomie du pays en matière d'intrants agricoles. La création de FOMI répond à un besoin stratégique national : réduire la dépendance du Burundi aux importations d'engrais et promouvoir la production locale à partir de matières premières organiques combinées.

La création de FOMI a coïncidé avec une décision gouvernementale stratégique : l'interdiction de l'importation d'engrais chimiques purs en 2019. Cette mesure, destinée à promouvoir l'industrie nationale et à préserver les réserves de devises, a conféré à la société une position de quasi-monopole sur le marché des engrais minéraux au Burundi. Aujourd'hui, l'entreprise demeure le fournisseur exclusif du Programme National de Subvention des Engrais au Burundi (PNSEB), produisant la totalité des engrais distribués dans le cadre de ce programme de subvention publique.

I.3 Situation géographique

L'usine principale est située à **Maramvya**, dans la Commune de **Mutimbuzi**, au sein de la nouvelle province de **Bujumbura**. Ce choix de localisation, à proximité immédiate de l'Aéroport International Melchior Ndadaye, du lac Tanganyika et des axes de communication majeurs, facilite à la fois l'importation des intrants minéraux complémentaires et

l'acheminement des produits finis vers l'intérieur du pays. La zone industrielle de Maramvya bénéficie d'une connexion routière directe, se situant à seulement **11 km du centre de la ville de Bujumbura** via la route nationale 5 (RN5) et à environ **100 km de Gitega**, la capitale politique.

L'implantation de l'usine à Maramvya s'explique par la proximité des axes de communication, du lac Tanganyika et de l'aéroport international Melchior Ndadaye. Cette localisation facilite à la fois l'importation des matières premières minérales complémentaires et l'acheminement des produits finis vers les dépôts provinciaux et zonaux du pays.

I.4 Vision, missions et objectifs de l'institution

FOMI a pour vision de devenir le leader de la production et de la distribution des engrais organo-minéraux dans la région des Grands Lacs, en garantissant aux agriculteurs burundais un accès permanent à des intrants de qualité, à un prix abordable et en quantité suffisante.

Ses missions consistent notamment à produire localement des engrais organo-minéraux, à sécuriser l'approvisionnement du PNSEB, à assurer la qualité et la traçabilité des produits, à réduire la dépendance du pays aux importations d'engrais et à contribuer à la sécurité alimentaire nationale.

Les objectifs poursuivis sont l'amélioration des rendements agricoles, la réduction des pertes, la professionnalisation du circuit de distribution, la digitalisation de la gestion des stocks et le renforcement de la transparence dans l'attribution des intrants subventionnés.

Le tableau ci-dessous présente les principaux produits commercialisés par FOMI.

Produit	Formule	Conditionnement	Usage principal
FOMI-IMBURA	NPK 9-22-4	25 kg	Céréales et cultures de base
FOMI-BAGARA	NK 11-0-22	25 kg	Tubercules et bananiers
FOMI-TOTAHAZA	NK 21-0-8	25 kg	Maraîchage et céréales
Chaux agricole	Amendement calcique	50 kg	Correction de l'acidité des sols

Tableau I.1 — Produits FOMI

I.5 Structure organisationnelle

FOMI est organisée autour d'une direction générale qui coordonne plusieurs directions et services complémentaires. La direction commerciale joue un rôle central dans la planification des ventes, la gestion des commandes et la relation avec les bénéficiaires du PNSEB. La direction de production assure la fabrication des engrais, tandis que le service qualité contrôle la conformité des lots. Le service logistique et distribution organise l'acheminement des produits de l'usine vers les dépôts provinciaux, zonaux et collinaires. Les services financier, des ressources humaines et informatique apportent leur appui transversal.

Direction / Service	Rôle principal
Direction générale	Pilotage stratégique et représentation de l'entreprise
Direction commerciale	Ventes, commandes, relation avec le PNSEB et les bénéficiaires
Direction de production	Fabrication des engrais et gestion des matières premières
Service qualité	Contrôle de conformité des lots et des produits finis
Service logistique et distribution	Acheminement et gestion des dépôts
Service financier	Trésorerie, facturation et suivi des subventions
Service des ressources humaines	Gestion du personnel et formation
Service informatique	Systèmes d'information et sécurité

Tableau I.2 — Structure organisationnelle simplifiée

I.6 Déroulement du stage

I.6.1 Introduction

Notre stage s'est déroulé au sein de la Direction Commerciale de FOMI. Encadrés par le Directeur Commercial, nous avons participé à l'analyse du circuit de distribution, à la collecte des besoins des utilisateurs et à la conception d'un système informatisé de gestion des stocks d'engrais. Le stage a alterné des périodes d'observation sur le terrain et des phases de travail technique au bureau.

I.6.2 Activités réalisées

— Immersion dans l'entreprise et entretiens avec le Directeur Commercial et les agents de distribution ;

— Observation du circuit de distribution, de l'usine de Maramvya jusqu'aux points de distribution collinaires ;

— Analyse des documents existants : registres papier, tableurs Excel, tickets de distribution et rapports ;

— Rédaction du cahier des charges : besoins fonctionnels et non fonctionnels, règles métier et cas d'utilisation ;

— Modélisation UML : cas d'utilisation, classes, séquences, activités et déploiement ;

— Développement de l'API REST avec Java 21 et Spring Boot 3.3.5 ;

— Conception du modèle de données PostgreSQL et des migrations Flyway ;

— Mise en place de la sécurité JWT, du contrôle d'accès par rôles et de la piste d'audit ;

— Écriture et exécution des tests d'intégration avec Testcontainers et JUnit 5 ;

— Développement de deux clients prototypes : une application web React et une application mobile Expo/React Native ;

— Documentation de l'API avec Swagger/OpenAPI et mise en place du pipeline d'intégration continue.

I.6.3 Étude de l'existant

L'étude de l'existant a consisté à décrire le fonctionnement actuel de la gestion des stocks et de la distribution, puis à en dégager les limites et à proposer une solution adaptée.

I.6.3.1 Description de l'existant

Avant la mise en place du système proposé, la gestion des stocks et des distributions reposait essentiellement sur des registres papier et des tableurs Excel décentralisés. Les mouvements étaient enregistrés manuellement par les agents, puis consolidés au niveau central avec des délais importants.

Le circuit de distribution des engrais

La distribution des engrais FOMI suit un circuit logistique et administratif hiérarchisé, depuis l'usine de Maramvya jusqu'aux agriculteurs bénéficiaires (Figure I.1). Ce circuit, restructuré en conformité avec l'organisation territoriale du pays, implique une chaîne d'approvisionnement en quatre niveaux de stockage fixes, complétée par un système de distribution finale au niveau de chaque colline.

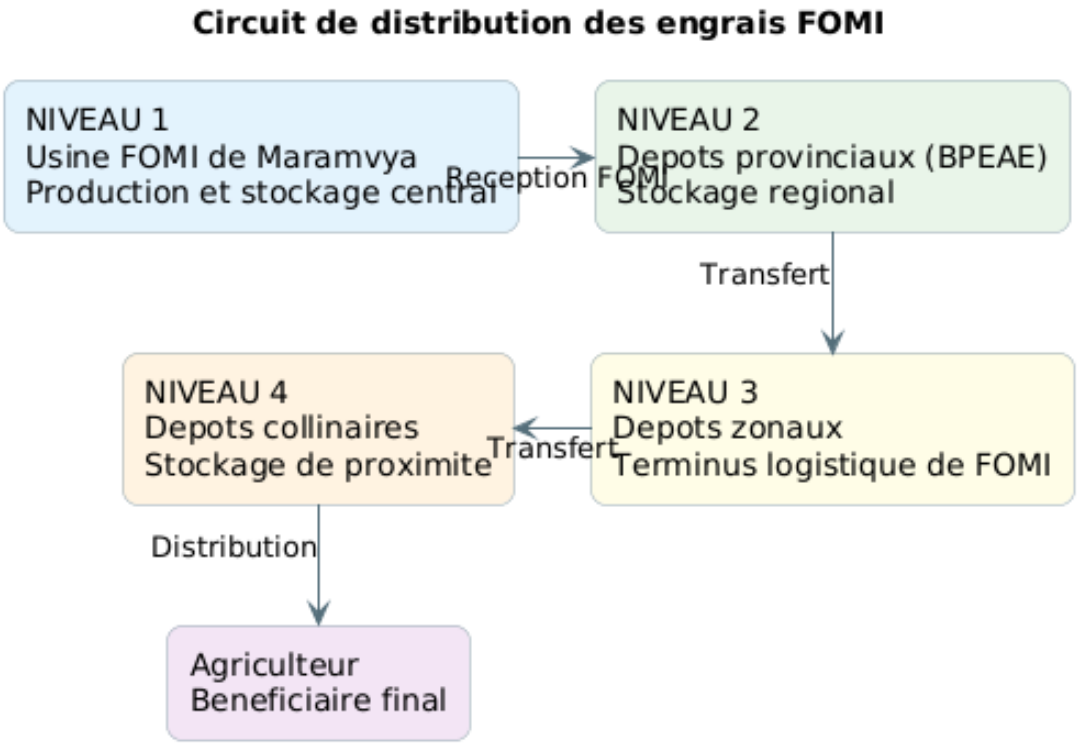


Figure I.1 — Circuit de distribution et d'acheminement des engrais FOMI

- **Niveau 1 : L'usine FOMI de Maramvya (Stockage Central)** Point de production et de stockage de référence. Après fabrication, les intrants sont entreposés dans des magasins couverts d'une capacité totale de 15 000 tonnes, dans l'attente des ordres de transfert. C'est également le point de contrôle qualité où chaque lot est analysé avant d'être pris en charge par l'agent d'usine pour les expéditions massives.
- **Niveau 2 : Les dépôts provinciaux — BPEAE (Stockage Régional)** Situés au niveau des structures des **Bureaux Provinciaux de l'Environnement, de l'Agriculture et de l'Élevage (BPEAE)**. Ces hubs logistiques reçoivent les livraisons de l'usine par gros porteurs et assurent la répartition des stocks entre les différentes communes et zones de leur ressort. Leur capacité varie de 500 à 2 000 tonnes selon la densité démographique agricole de la province.
- **Niveau 3 : Les dépôts de zone (Terminus logistique officiel de la FOMI)** C'est le dernier point de stockage fixe et permanent maîtrisé par l'usine. Chaque zone dispose d'un magasin d'entrepôt (d'une capacité de 100 à 500 tonnes) où les stocks sont

acheminés et comptabilisés. Les agents de zone y valident la réception des volumes physiques et centralisent le suivi des stocks intermédiaires.

- **Niveau 4 : Les points de distribution de colline (Livraison éphémère)** À ce stade, la logistique lourde s'arrête. Sous la coordination de l'administration locale et des comités collinaires, les engrais sont transférés depuis le dépôt de zone vers les collines de destination. Faute d'infrastructures de stockage permanentes, des sites communautaires éphémères (hangars, églises, écoles) sont réquisitionnés uniquement le jour de la distribution pour décharger temporairement les volumes (20 à 100 tonnes) et servir immédiatement les producteurs.
- **Le bénéficiaire final : L'agriculteur** L'exploitant se présente au point de distribution collinaire muni de sa pièce d'identité et de son bon de commande numérique ou papier (ticket de distribution). Ce ticket nominatif, délivré après paiement, spécifie la nature de l'engrais organo-minéral, le quota exact autorisé calculé selon ses superficies, ainsi que la saison agricole concernée.

Le processus de distribution s'appuie sur une mécanique financière stricte. Les institutions financières partenaires, principalement la **Régie Nationale des Postes** (la Poste du Burundi) et la **FENACOBU**, interviennent comme intermédiaires pour collecter les fonds. Un acompte de 50 % du prix subventionné est exigé lors de l'inscription pour valider la commande, générant l'avance de trésorerie nécessaire à l'importation des matières premières minérales complémentaires en devises étrangères. Le solde restant de 50 % est versé par l'agriculteur juste avant le retrait effectif de ses intrants.

Les dysfonctionnements observés sur le terrain

Malgré ses réalisations, la société FOMI fait face à des défis logistiques et financiers majeurs qui entravent l'efficacité de la distribution. Ces anomalies, structurellement accentuées par l'absence d'un système d'information unifié et transparent pour la traçabilité des stocks, constituent le cœur de la problématique de ce travail de recherche.

- **Ruptures de stock et décalages saisonniers :** Bien que l'usine maintienne une production à grande échelle, la chaîne d'approvisionnement subit des interruptions chroniques liées aux tensions sur les devises étrangères requises pour l'achat des intrants minéraux. Face à une demande nationale consolidée par le gouvernement à plus de **164 216 tonnes d'engrais** pour la campagne, les retards de livraison aux dépôts zonaux s'étendent parfois sur plusieurs semaines. Selon les stations de recherche de l'**ISABU**, ce décalage forcé entre la période optimale de semis et l'application effective des nutriments engendre une baisse directe des rendements agricoles de l'ordre de 15 à 25 %.
- **Spéculation et opacité du marché noir :** Les distorsions de prix créent de fortes incitations au détournement d'intrants. Alors que le prix subventionné par l'État d'un sac de FOMI-IMBURA (25 kg) est réglementé à **31 000 FBu**, la rareté temporaire propulse les tarifs sur le marché informel entre 250 000 et 350 000 FBu pour les engrais azotés comme l'Urée. Ces fuites de stocks, matérialisées par des détournements nocturnes dans des zones rizicoles comme Gihanga ou des distributions fictives basées sur des listes collinaires gonflées, privent les petits exploitants de leurs droits d'accès.
- **Gouvernance budgétaire et traçabilité des subventions : Les failles du système de suivi manuel se répercutent jusqu'aux finances publiques. Fin 2025, un différend**

financier majeur a éclaté, la FOMI réclamant plus de 74 milliards de FBu au Trésor public pour des livraisons exécutées au-delà des volumes initialement autorisés par les contrats de subvention. Ce conflit, qualifié d'indiscipline budgétaire par le Ministère des Finances, démontre l'incapacité des outils actuels à plafonner automatiquement les flux de sortie et à lier strictement les facturations aux quotas réels des bénéficiaires. Ces décalages entraînent également des retards de paiement des bénéficiaires.

Ces difficultés financières chroniques compromettent directement la capacité de la société FOMI à investir de manière autonome dans l'extension de ses infrastructures de production ou dans la modernisation de ses équipements de suivi.

- **Irrégularités et opacité des distributions locales :** Les investigations de terrain publiées par le média IWACU en février 2026 ont mis en lumière d'importants dysfonctionnements administratifs au niveau collinaire, incluant des signalements de favoritisme et des volumes distribués non documentés. La frustration des exploitants est exacerbée par le phénomène des arriérés de livraison : de nombreux ménages ruraux ayant versé leurs avances financières depuis plus de deux ans attendent toujours la réception physique de leurs intrants. À Cibitoke, des producteurs ont rapporté n'avoir perçu qu'une fraction infime (quelques centaines de kilogrammes) des tonnes de fertilisants initialement facturées et pourtant payées au tarif subventionné. Cette rupture locale pousse les exploitants lésés à s'approvisionner sur le marché parallèle informel, où le prix spéculatif moyen oscille entre 250 000 et 350 000 FBu selon la rareté saisonnière.
- **Causalité systémique et justification du projet :** L'ensemble de ces défis converge vers un constat structurel central : l'absence d'un système d'information centralisé, sécurisé et auditable constitue le facteur aggravant majeur de tous les dysfonctionnements observés tout au long de la chaîne d'approvisionnement. Sans visibilité en temps réel sur les niveaux de stock par magasin de zone, sans traçabilité algorithmique des mouvements inter-dépôts et sans mécanisme automatisé d'alerte de rupture de stock, il s'avère impossible pour les autorités de régulation de détecter les flux illicites, de prévenir les distributions fictives et d'optimiser l'acheminement des intrants. C'est précisément l'urgence de pallier ces failles de gouvernance logistique qui motive la conception et le déploiement de l'API de gestion des stocks formalisée dans ce mémoire.

I.6.3.2 Critique de l’existant

La critique du système existant met en évidence plusieurs limites structurelles : absence de visibilité en temps réel, multiplication des supports, risques d’erreurs et de pertes d’information, difficulté d’audit et de contrôle des quotas. Les conséquences pour les parties prenantes sont détaillées ci-après.

Les anomalies documentées dans la section précédente se répercutent lourdement sur les trois acteurs pivots du système : les exploitants agricoles, la société FOMI et l'État burundais. Ces conséquences interconnectées alimentent un cercle vicieux qui fragilise l'ensemble de la chaîne de valeur nutritionnelle du pays.

- **Pour les agriculteurs : Les impacts sont directs et affectent la sécurité alimentaire de subsistance. Les ruptures physiques dans les magasins de zone et les retards chroniques d'acheminement privent les sols d'intrants au moment critique des semis, écrasant les rendements finaux. Les producteurs ayant versé leurs acomptes financiers auprès de la Poste ou de la FENACOBU sans percevoir leurs sacs se retrouvent piégés dans une détresse financière. Pour sauver leur saison, beaucoup se tournent vers le marché noir informel où les prix spéculatifs s'avèrent prohibitifs. Cette opacité généralisée érode la confiance des ménages ruraux envers le dispositif d'encadrement public.**
- **Pour la société FOMI : Les défaillances de traçabilité se traduisent par des pertes d'exploitation sèches adossées aux vols de lots et aux détournements de cargaisons. Les polémiques médiatiques récurrentes nuisent à l'image institutionnelle de l'entreprise, indispensable à ses ambitions d'exportation régionale. De plus, l'asymétrie de données consolidées sur les volumes réellement distribués au niveau collinaire empêche un calibrage fin des cycles industriels, provoquant des surstocks sur certaines gammes et des pénuries structurelles sur d'autres. Les tensions de trésorerie actuelles de l'usine découlent directement de ces frottements logistiques et des retards de recouvrement des subventions.**
- **Pour l'État burundais :** Le manque de visibilité en temps réel neutralise l'efficacité socio-économique du PNSEB, un effort financier stratégique qui capte **plusieurs centaines de milliards de FBu** sur les budgets du Ministère de l'Agriculture. Les fraudes documentées altèrent la crédibilité du programme auprès des bailleurs de fonds internationaux qui soutiennent le développement rural burundais. Enfin, l'incapacité de l'appareil d'État à honorer équitablement la distribution d'engrais payés d'avance génère de vives frustrations populaires au sein du monde rural, transformant un enjeu logistique en un facteur de risques pour la paix sociale

I.6.3.3 Solution proposée

La solution proposée est un système d'information centralisé composé de trois briques complémentaires : une API REST sécurisée qui porte la logique métier et la persistance, une application web destinée aux administrateurs et superviseurs, et une application mobile destinée aux agents de terrain. L'API couvre la gestion des référentiels (engrais, géographie, dépôts, agriculteurs), les réceptions et transferts de stock, la distribution FIFO, les tickets et leur workflow de validation, les alertes automatiques, les statistiques et la piste d'audit.

Le système retenu repose sur une API REST organisée en domaines fonctionnels, une base de données PostgreSQL, des migrations versionnées avec Flyway et une authentification par jetons. Il intègre également une documentation OpenAPI et une suite de tests d'intégration exécutés sur une base PostgreSQL réelle.

I.6.4 Corrélation du stage avec les Éléments Constitutifs des Unités d'Enseignement

Le tableau ci-dessous met en relation les activités réalisées durant le stage avec les éléments constitutifs des unités d'enseignement (ECU) suivies au cours de notre formation.

Élément constitutif d'UE	Contenu mobilisé	Application dans le stage
Programmation orientée objet	Classes, objets, encapsulation, héritage, polymorphisme	Développement de l'API en Java
Bases de données	Modèle relationnel, normalisation, SQL, transactions	Modèle PostgreSQL, migrations Flyway
Génie logiciel	Cycle de vie, UML, analyse des besoins, tests	Cahier des charges, diagrammes, tests
Architecture logicielle	Architecture trois tiers, API REST, design patterns	Architecture trois tiers de l'API
Sécurité informatique	Authentification, autorisation, hachage, JWT	Sécurité de l'API et contrôle d'accès
Réseaux et systèmes distribués	Protocoles HTTP, services web, client-serveur	API REST et clients web/mobile
Méthodologie de recherche	Entretiens, observation, analyse documentaire	Collecte des besoins sur le terrain
Gestion de projet	Planification, suivi, gestion des	Organisation du travail et Git/GitHub

Élément constitutif d’UE	Contenu mobilisé	Application dans le stage
	versions	

Tableau I.3 — Corrélation stage / ECU

CHAPITRE II : CONCEPTS THÉORIQUES ET MÉTHODOLOGIE DE L'ÉTUDE

Ce chapitre présente les concepts théoriques sur lesquels repose notre travail ainsi que la méthodologie adoptée pour mener l'étude. Les concepts couvrent la gestion des stocks, la traçabilité, l'architecture logicielle, la sécurité et les technologies utilisées. La méthodologie précise la démarche, les phases du projet, les outils et les modalités de validation.

II.1 Concepts théoriques

II.1.1 Gestion des stocks et méthode FIFO

La gestion des stocks consiste à suivre en permanence les quantités disponibles par produit, par dépôt et par lot, afin de garantir la disponibilité des intrants et d'éviter aussi bien les ruptures que les surstocks. La méthode FIFO (First-In, First-Out) consiste à consommer en priorité les lots les plus anciens. Dans le cas des engrais, dont la durée de conservation est limitée, ce principe doit être combiné à un contrôle de la date de péremption afin de distribuer d'abord les lots les plus proches de leur date limite. Le système retenu applique ce principe lors des sorties de stock et enregistre chaque décrément sous forme de mouvement traçable.

II.1.2 Traçabilité et piste d'audit

La traçabilité est la capacité à reconstituer, pour chaque lot et chaque ticket, l'ensemble des opérations effectuées, leur auteur et leur date. Elle repose sur une piste d'audit (audit trail) qui enregistre les actions critiques dans une table dédiée : création, validation, distribution, annulation et modification des référentiels. Une piste d'audit fiable est indispensable dans un dispositif subventionné, car elle permet de contrôler les quotas, de détecter les anomalies et de répondre aux demandes des autorités de tutelle.

II.1.3 Technologies et outils utilisés

II.1.3.1 Java 21 et Spring Boot 3.3.5

Java a été choisi comme langage de programmation principal pour sa maturité (30 ans d'existence), sa portabilité (Write Once, Run Anywhere) et son écosystème riche en bibliothèques et frameworks d'entreprise. La version 21, sortie en septembre 2023, est une version LTS (Long-Term Support) qui offre des améliorations significatives en termes de performance (Virtual Threads pour la gestion efficace des threads), de sécurité (updated cryptography libraries) et de productivité (pattern matching, record patterns, sequenced collections).

Spring Boot 3.3.5, basé sur le framework Spring (version 6.1), a été retenu pour sa capacité à simplifier le développement d'applications Java d'entreprise. Le principe de convention over configuration réduit considérablement la quantité de code boilerplate nécessaire, permettant aux développeurs de se concentrer sur la logique métier. Spring Boot intègre Tomcat comme serveur HTTP intégré, simplifiant le déploiement. Les fonctionnalités clés utilisées incluent : Spring MVC pour le routage REST, Spring Data JPA pour la persistance, Spring Security pour la sécurité, et Spring Scheduling pour les cron jobs.

II.1.3.2 PostgreSQL 16

PostgreSQL 16 a été sélectionné comme SGBD pour sa robustesse éprouvée, sa conformité aux standards SQL et ses fonctionnalités avancées. Les énumérations nommées offrent un mécanisme natif pour garantir l'intégrité des données au niveau de la base. Le type JSONB permet de stocker des données structurées de manière flexible, notamment les détails des opérations dans la table audit_log. Les index B-tree et composites sur les colonnes fréquemment utilisées dans les clauses WHERE, JOIN et ORDER BY réduisent significativement le temps d'exécution des requêtes. Aucun index GIN n'est utilisé dans l'implémentation actuelle.

II.1.3.3 Flyway — Gestion des migrations

Flyway a été choisi pour la gestion versionnée des évolutions du schéma de la base de données. Chaque modification du schéma (ajout de table, modification de colonne, ajout d'index, création d'enum) est encapsulée dans un fichier de migration SQL numéroté

(V1__init.sql, V2__add_alertes.sql, etc.). Flyway maintient une table d'historique (flyway_schema_history) permettant de tracer l'ensemble des évolutions du schéma depuis la création du projet. Le projet compte actuellement 12 versions de migration, assurant la reproductibilité du schéma sur n'importe quel environnement (développement, test, production).

II.1.3.4 Spring Security et JWT

Spring Security est le framework de référence pour la sécurité des applications Spring. Dans le contexte de l'API FOMI, il est configuré en mode stateless, désactivant les sessions côté serveur. Le cycle JWT complet est implémenté : génération du token lors du login, validation à chaque requête via un filtre personnalisé, et rejet automatique des tokens expirés. Les règles d'autorisation sont déclarées dans la classe SecurityConfig, sous forme de règles par URL et par méthode HTTP ; le code n'utilise pas les annotations @PreAuthorize ou @Secured.

II.1.3.5 Spring Data JPA et Hibernate

Spring Data JPA, avec Hibernate 6 comme implémentation, assure la couche de persistance. Le pessimistic locking via SELECT FOR UPDATE est utilisé dans StockLotRepository pour les opérations de décrémentation de stock, évitant les conditions de course en présence de transactions concurrentes. Les requêtes JPQL dynamiques et les JPA Specifications permettent d'implémenter la recherche multicritère avec des filtres variables tout en maintenant la sécurité contre les injections SQL.

II.1.3.6 Testcontainers et JUnit 5

Testcontainers lance une instance PostgreSQL réelle en Docker pour les tests d'intégration, garantissant la fidélité des tests par rapport à la production. JUnit 5 fournit les annotations et les assertions. Les 218 tests se répartissent en 215 tests d'intégration (PostgreSQL) et 3 tests unitaires de configuration ; ils couvrent le contrôle d’accès, la concurrence (pessimistic locking) et les cas métier (FIFO, annulation, alertes).

II.1.3.7 SpringDoc OpenAPI — Documentation

SpringDoc OpenAPI génère automatiquement la documentation Swagger/OpenAPI 3.0 à partir des annotations @Operation, @Parameter, @Schema du code. L'interface Swagger UI,

accessible via /swagger-ui.html, permet aux développeurs de tester les endpoints en temps réel et de générer automatiquement les clients API dans différents langages.

II.1.3.8 Lombok

Lombok réduit le code boilerplate via les annotations @Data, @Builder, @NoArgsConstructor, @AllArgsConstructor, @Slf4j, appliquées au niveau des entités, DTOs et services. Cette réduction de la verbose du code Java améliore la lisibilité et réduit le risque d'erreurs liées à la maintenance de code répétitif.

Tableau II.1 — Comparatif des choix technologiques

Technologie	Rôle	Justification
Java 21	Langage	LTS, typé statiquement, performance, Virtual Threads, écosystème riche
Spring Boot 3.3.5	Framework	Convention over configuration, intégration complète, communauté active
PostgreSQL 16	SGBD	Enums nommés, JSONB, index B-tree et composites, performance, open source, ACID
Flyway	Migrations	Versionné, traçable, intégré Spring Boot, reproductible
Spring Security	Sécurité	JWT stateless, BCrypt, autorisation par URL
Spring Data JPA	Persistance	Pessimistic locking, JPQL dynamique, JPA Specifications
Testcontainers	Tests intégration	PostgreSQL réel en Docker, fidélité production
JUnit 5	Tests	Standard Java, @Nested, @ParameterizedTest, assertions riches
SpringDoc	Documentation	Swagger auto-généré, interactif, export OpenAPI JSON
Lombok	Utilitaires	Réduction boilerplate, @Data, @Builder, @Slf4j

II.1.4 État de l’art : systèmes de référence en Afrique de l’Est

Plusieurs pays de la Communauté d'Afrique de l'Est (EAC) ont amorcé la transformation numérique de leur gouvernance agricole, offrant des architectures de référence pour le projet FOMI. L'analyse comparative de ces écosystèmes permet d'identifier les meilleures pratiques logistiques et d'asseoir la pertinence fonctionnelle de notre API.

- **Le modèle d'intégration du Rwanda : Le Smart Nkunganire System (SNS) : Pour l'administration des intrants subventionnés, le Rwanda a déployé à grande échelle le Smart Nkunganire System (SNS), une plateforme centralisée gérant plus de 1,9 million de producteurs interconnectés aux coopératives et agro-dealers. Ce système valide en temps réel les quotas d'engrais éligibles par exploitation. Le cas rwandais est une référence majeure pour le Burundi en raison de caractéristiques territoriales similaires ; le SNS a prouvé qu'un suivi rigoureux des stocks atténuait structurellement les goulets d'étranglement de la chaîne de distribution.**
- **Le modèle d'allocation du Kenya : Le système d'e-Voucher du NFSP : Le Ministère de l'Agriculture du Kenya, en synergie avec l'opérateur Safaricom, s'appuie sur le National Fertilizer Subsidy Program matérialisé par des bons d'achat électroniques (e-Vouchers). Via une interface USSD (*707#), l'agriculteur accède à son portefeuille numérique, lui permettant de régler sa quote-part auprès des dépôts céréaliers (NCPB) et des agrovets privés. Ce système démontre l'intérêt d'automatiser la réconciliation financière en temps réel pour sécuriser les flux de paiement et endiguer la spéculation du marché informel.**
- **Le modèle de contrôle de la Tanzanie : Le Fertilizer Information System (FIS) : La Tanzania Fertilizer Regulatory Authority (TFRA) utilise le Fertilizer Information System (FIS), un portail unifié qui centralise l'enregistrement de l'historique des lots importés ou fabriqués localement, l'octroi des licences commerciales et le profilage numérique des bénéficiaires. L'approche tanzanienne souligne l'importance d'intégrer des fonctionnalités de conformité réglementaire dès la conception de l'API pour certifier la qualité des flux d'intrants circulant dans le pays.**

L'analyse de ces initiatives régionales révèle une convergence technologique vers des besoins fondamentaux : suivi centralisé des volumes, dématérialisation des flux financiers et traçabilité des droits des exploitants. Ces constats valident l'orientation technique choisie pour l'API de gestion du système FOMI, tout en rappelant la nécessité de l'adapter aux spécificités burundaises, notamment une architecture logistique articulée autour de quatre niveaux de stockage fixes et un système de subvention intermédié par les réseaux bancaires nationaux.

II.2 Méthodologie de l'étude

II.2.1 Démarche méthodologique

Nous avons adopté une démarche qualitative et participative articulée autour de quatre axes : l'analyse documentaire, les entretiens semi-directifs, l'observation directe et la modélisation. L'analyse documentaire a porté sur les rapports internes, les registres de stock, les tableurs et les tickets. Les entretiens ont été menés auprès du Directeur Commercial et des agents de distribution afin de comprendre les besoins réels et les difficultés opérationnelles. L'observation directe nous a permis de suivre le circuit des engrais et d'identifier les goulets d'étranglement. Enfin, la modélisation UML a servi de support de communication entre les besoins exprimés et la solution technique.

II.2.2 Phases du projet

— Phase 1 — Analyse de l'existant : compréhension du circuit, des acteurs et des documents ;

— Phase 2 — Analyse des besoins : identification des besoins fonctionnels et non fonctionnels, des règles métier et des cas d'utilisation ;

— Phase 3 — Conception : architecture, modèle de données, diagrammes UML et choix technologiques ;

— Phase 4 — Implémentation : développement de l'API, des clients web et mobile, de la sécurité et de l'audit ;

— Phase 5 — Tests : tests d'intégration, tests de sécurité et validation fonctionnelle ;

— Phase 6 — Déploiement et documentation : conteneurisation, déploiement, documentation Swagger et rédaction du rapport.

II.2.3 Outils de développement et de validation

Les principaux outils utilisés sont : Java 21, Spring Boot 3.3.5, Maven, PostgreSQL 16, Flyway, Spring Security et JJWT, Spring Data JPA/Hibernate, SpringDoc OpenAPI, JUnit 5, Testcontainers, Git et GitHub, Docker, Railway, React/Vite et Expo/React Native pour les clients, ainsi que Swagger UI et Postman pour la validation des endpoints.

II.2.4 Méthode de validation des résultats

La validation repose sur plusieurs niveaux : la revue du cahier des charges avec le maître de stage, la vérification de la conformité des endpoints aux besoins, l'exécution des tests d'intégration sur une base PostgreSQL réelle via Testcontainers, les tests de contrôle d'accès et l'intégration continue avec GitHub Actions. Les résultats sont présentés au chapitre III.

CHAPITRE III : PRÉSENTATION DES RÉSULTATS

Ce chapitre présente les résultats du travail : l'analyse des besoins, la conception du système, l'implémentation, les tests et le déploiement, l'évaluation des résultats ainsi que les clients web et mobile développés.

III.1 Analyse des besoins

Cette section présente l'analyse détaillée des besoins fonctionnels et non fonctionnels du système, les règles métier à implémenter et les cas d'utilisation identifiés lors de la phase d'analyse du projet. Cette analyse constitue la base contractuelle du développement, définissant précisément ce que le système doit faire et les contraintes auxquelles il doit se conformer. Les besoins ont été recueillis à travers des entretiens avec les responsables de FOMI et l'écoute des plaintes des agriculteurs.

III.1.1 Besoins fonctionnels

Vingt-cinq besoins fonctionnels ont été identifiés à partir de l'analyse des exigences de FOMI, des entretiens avec les parties prenantes et de l'étude du contexte opérationnel. Ces besoins couvrent l'authentification et la sécurité, la gestion des données de référence, la gestion des stocks, la distribution et les tickets, le reporting et l'audit, ainsi que l'intégrité des données.

Tableau III.1 — Besoins fonctionnels détaillés

ID	Catégorie	Description
F1	Authentification	Authentification JWT avec contrôle d'accès par rôle
F2	Données référence	Gestion des types d'engrais (CRUD) avec composition et seuils d'alerte
F3	Données référence	Gestion des provinces/zones/collines (hiérarchie administrative)
F4	Données référence	Gestion des dépôts (CRUD, liaison hiérarchique, capacité de stockage)
F5	Gestion agriculteurs	CRUD des agriculteurs avec identifiant unique, superficie, colline
F6	Tickets	Création et suivi des tickets de distribution avec workflow

ID	Catégorie	Description
F7	Stocks	Réception FOMI (entrées de stock)
F8	Stocks	Transferts inter-dépôts (province, zone, colline) avec vérification de disponibilité à la source
F9	Distribution	Distribution aux agriculteurs selon l'algorithme FIFO
F10	Stocks	Gestion des lots (dates de fabrication et de péremption)
F11	Stocks	Suivi des stocks par dépôt, type d'engrais et lot
F12	Alertes	Système d'alertes automatiques (rupture, stock bas, retard, péremption, inactivité)
F13	Mouvements	Annulation de mouvements (transfert avec rétroaction stock, distribution avec mise à jour du ticket)
F14	Tickets	Validation et rejet des tickets (workflow multi-états avec motifs)
F15	Reporting	Tableaux de bord analytiques (taux de service, stocks régionaux)
F16	Reporting	Export de données au format CSV (tickets et mouvements)
F17	Audit	Piste d'audit avec détails JSON
F18	Alertes	Configuration des seuils d'alerte par dépôt et type d'engrais
F19	Transparence	Consultation publique des tickets (sans authentification)
F20	Recherche	Recherche multicritère sur les agriculteurs et les lots avec pagination côté serveur
F21	Reporting	Extraction des volumes non servis par colline
F22	Reporting	Calcul du taux de service par province et par zone
F23	Reporting	Journalisation des mouvements par agent
F24	Sécurité	Gestion des comptes utilisateurs et des rôles (CRUD par admin)
F25	Intégrité	Cohérence des stocks et des tickets (contraintes SQL, transactions, anti-double annulation)

III.1.2 Règles métier

Vingt-deux règles métier ont été identifiées pour garantir l'intégrité des données et la cohérence des opérations dans le système. Ces règles encadrent les opérations de stock, la distribution aux agriculteurs, la validation des tickets et le fonctionnement du système d'alertes. Chaque règle est implémentée dans les services du système et validée par les tests d'intégration correspondants.

Tableau III.2 — Règles métier

ID	Catégorie	Règle
R1	Intégrité stock	Le stock d'un dépôt ne peut pas devenir négatif (contrainte CHECK quantite_disponible >= 0).
R2	Quota distribution	La quantité d'un ticket ne peut excéder la superficie × 150 kg/ha pour le type d'engrais demandé.
R3	Seuil alerte	Une alerte est levée lorsque le stock disponible est inférieur au seuil configuré pour un dépôt et un type d'engrais.
R4	Péremption	Les lots dont la date de péremption est dépassée sont exclus de la distribution FIFO ; le chemin avec lot imposé doit encore être corrigé.
R5	Capacité dépôt	La somme des stocks ne doit pas dépasser la capacité maximale du dépôt.
R6	Unicité ticket	Impossible de créer deux tickets pour la même saison, le même type d'engrais et le même agriculteur.
R7	Annulation	Un mouvement déjà annulé ne peut pas faire l'objet d'une seconde annulation.
R8	Rétroaction	L'annulation d'un transfert restaure le stock au dépôt source et le retire du dépôt destination ; l'annulation d'une distribution met à jour le ticket et crée un ajustement.
R9	Transfert	Tout transfert inter-dépôts valide d'abord la disponibilité réelle des lots dans le dépôt source.
R10	Validation ticket	Un ticket peut être validé ou rejeté avec un motif obligatoire.
R11	FIFO	L'allocation des engrais suit un algorithme FIFO basé sur la date de réception FOMI, après exclusion des lots périmés.
R12	Capacité max	La capacité maximale du dépôt est vérifiée lors d'une réception.

ID	Catégorie	Règle
R13	Audit	Chaque opération critique génère un log d'audit via AuditLogService.
R14	Rôle admin	Seul le profil ADMIN dispose des droits CRUD pour gérer les comptes utilisateurs et configurer les seuils d'alerte.
R15	Périmètre	Un utilisateur AGENT_ZONE est limité aux données et mouvements des dépôts de sa zone.
R16	Export	L'extraction CSV des tickets est plafonnée à 100 000 lignes ; l'export des mouvements n'est pas plafonné.
R17	Inactivité	Une alerte est générée si un dépôt actif n'enregistre aucun mouvement durant 7 jours consécutifs.
R18	Recherche	Les endpoints de recherche multicritère doivent implémenter une pagination côté serveur.
R19	JWT	Un token expiré (24 h) est rejeté automatiquement avec le code 401.
R20	Ticket expiré	Un ticket non servi peut être marqué EXPIRE via TicketService.expirerTickets(), sans planification automatique.
R21	Quota saison	Le contrôle des droits aux engrais subventionnés est réinitialisé et cloisonné par saison agricole.
R22	BCrypt	Les mots de passe sont hashés avec BCrypt (cost 10).

III.1.3 Cas d’utilisation

Vingt-six cas d'utilisation ont été identifiés pour modéliser les interactions entre les acteurs du système et les fonctionnalités de l'application. Chaque cas d'utilisation est décrit avec son identifiant, les acteurs concernés, les préconditions et le scénario principal. Les acteurs identifiés sont : ADMIN (administrateur système), SUPERVISEUR (responsable provincial ou de zone), AGENT_COLLINE (agent de distribution au niveau colline), AGENT_ZONE (agent de distribution au niveau zone), AGENT_LOGISTIQUE (agent de réception à l'usine), PUBLIC (consultation sans authentification) et TOUT_UTILISATEUR (n'importe quel utilisateur authentifié).

Tableau III.3 — Cas d'utilisation

ID	Cas d'utilisation	Acteur	Précondition	Résultat
UC-01	Authentification	Tout utilisateur	Identifiants valides	Génération et retour d'un jeton JWT
UC-02	Gérer types engrais	ADMIN	Authentifié	Opérations CRUD sur les formules (NPK, Urée)
UC-03	Gérer géographie	ADMIN	Authentifié	CRUD provinces/zones/collines
UC-04	Gérer dépôts	ADMIN	Géographie définie	CRUD des dépôts (province, zone, colline)
UC-05	Gérer agriculteurs	ADMIN/ AGENT_ZONE	Authentifié	Établissement du registre des exploitants
UC-06	Créer ticket	AGENT_ZONE	Agriculteur identifié	Génération d'un ticket en état EN_ATTENTE_DE_VALIDATION
UC-07	Valider/Rejeter ticket	SUPERVISEUR	Ticket en attente	Ticket actif ou rejeté
UC-08	Réceptionner engrais	AGENT_LOGISTIQUE	Livraison reçue	Mise à jour des stocks du dépôt récepteur
UC-09	Transférer dépôts	ADMIN	Stock source dispo	Transfert exécuté
UC-10	Distribuer agriculteurs	AGENT_ZONE / AGENT_COLLINE	Ticket actif	Déstockage automatique via l'algorithme FIFO
UC-11	Gérer lots	ADMIN	Type engrais défini	Suivi des numéros de lots, fabrications et péremptions
UC-12	Annuler mouvement	UTILISATEUR AUTHENTIFIÉ	Mouvement identifié	Annulation + ajustement
UC-13	Consulter stocks	Tout authentifié	Périmètre défini	Affichage filtré des volumes physiques
UC-14	Consulter alertes	SUPERVISEUR	Alertes existantes	Visualisation des alertes de rupture

ID	Cas d'utilisation	Acteur	Précondition	Résultat
				ou péremption
UC-15	Consulter stats	SUPERVISEUR	Données disponibles	Tableaux de bord
UC-16	Exporter CSV	ADMIN / SUPERVISEUR / AGENT	Données existantes	Fichier CSV généré
UC-17	Consulter audit logs	ADMIN	Logs existants	Historique filtré
UC-18	Ticket public	PUBLIC	Identifiant ticket	Restitution anonyme du statut d'éligibilité du bon
UC-19	Gérer utilisateurs	ADMIN	Droits admin	CRUD utilisateurs
UC-20	Gérer seuils alerte	ADMIN	Type engrais défini	Seuils configurés
UC-21	Rechercher stocks	AGENT	Critères recherche	Résultats paginés
UC-22	Rechercher agriculteurs	AGENT	Critères recherche	Résultats paginés
UC-23	Mouvements par lot	AGENT	Lot identifié	Historique mouvements
UC-24	Filtrer mouvements	AGENT	Filtres disponibles	Mouvements filtrés
UC-25	Taux service	SUPERVISEUR	Province/zone définie	Calcul automatique du ratio commandes / livraisons.
UC-26	Tickets non servis	SUPERVISEUR	Zone définie	Liste tickets en attente

III.1.4 Besoins non fonctionnels

Dix besoins non fonctionnels ont été identifiés pour garantir la qualité, la performance et la sécurité du système. Ces exigences transcend les fonctionnalités métier pour définir les propriétés de qualité du système en tant qu'infrastructure logicielle.

Tableau III.4 — Besoins non fonctionnels

ID	Catégorie	Exigence
NF1	Performance	Temps de réponse < 200ms pour 95% des requêtes de lecture
NF2	Disponibilité	Disponibilité cible ≥ 99,5 % sur une période annuelle
NF3	Sécurité	Authentification JWT avec tokens HMAC-SHA expirant après 24h
NF4	Scalabilité	Architecture stateless permettant la montée en charge horizontale
NF5	Documentation	API documentée via Swagger/OpenAPI 3.0 accessible en ligne
NF6	Tests	Suite de tests automatisés avec couverture de code > 80%
NF7	Déploiement	Pipeline CI/CD avec déploiement continu via GitHub Actions
NF8	Audit	Enregistrement de toutes les opérations critiques avec horodatage
NF9	Multi-niveaux	Filtrage périmètre par niveau hiérarchique géographique
NF10	Sauvegarde	Sauvegardes automatiques quotidiennes de la base PostgreSQL

Le diagramme de cas d'utilisation (Figure III.1) présente les 26 cas d'utilisation principaux (UC-01 à UC-26) avec leurs acteurs et relations include/extend, modélisant l'ensemble des interactions entre utilisateurs et système.

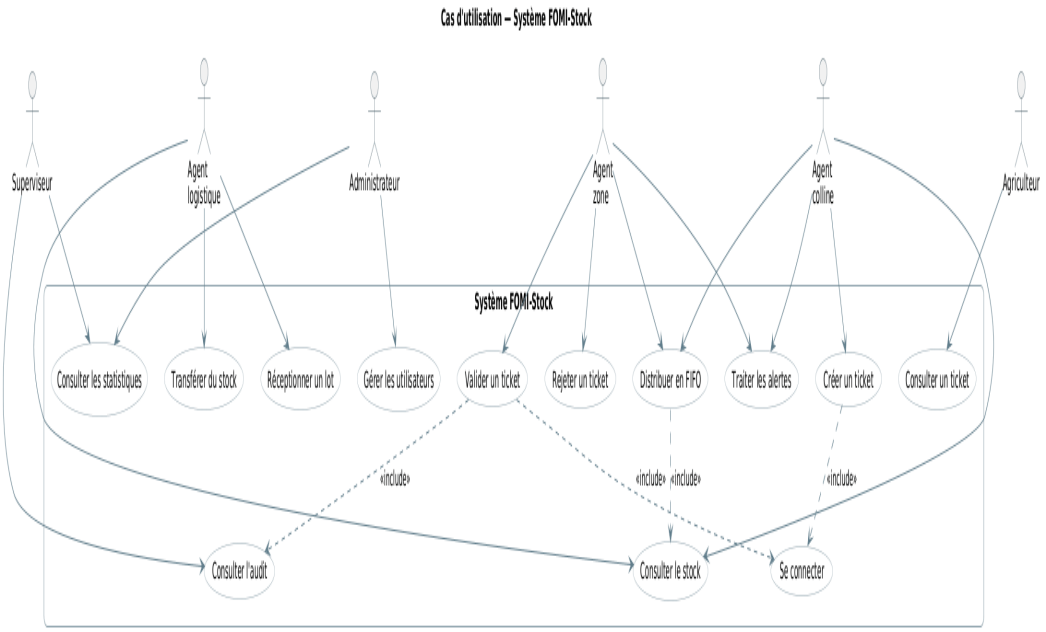


Figure III.1 — Diagramme de cas d'utilisation UML

Le diagramme d'activités (Figure III.2) modélise le workflow de validation et de distribution d'un ticket sous forme d'activités et de transitions conditionnelles, montrant le parcours depuis la vérification du périmètre agent jusqu'à la création des mouvements et la génération des alertes.

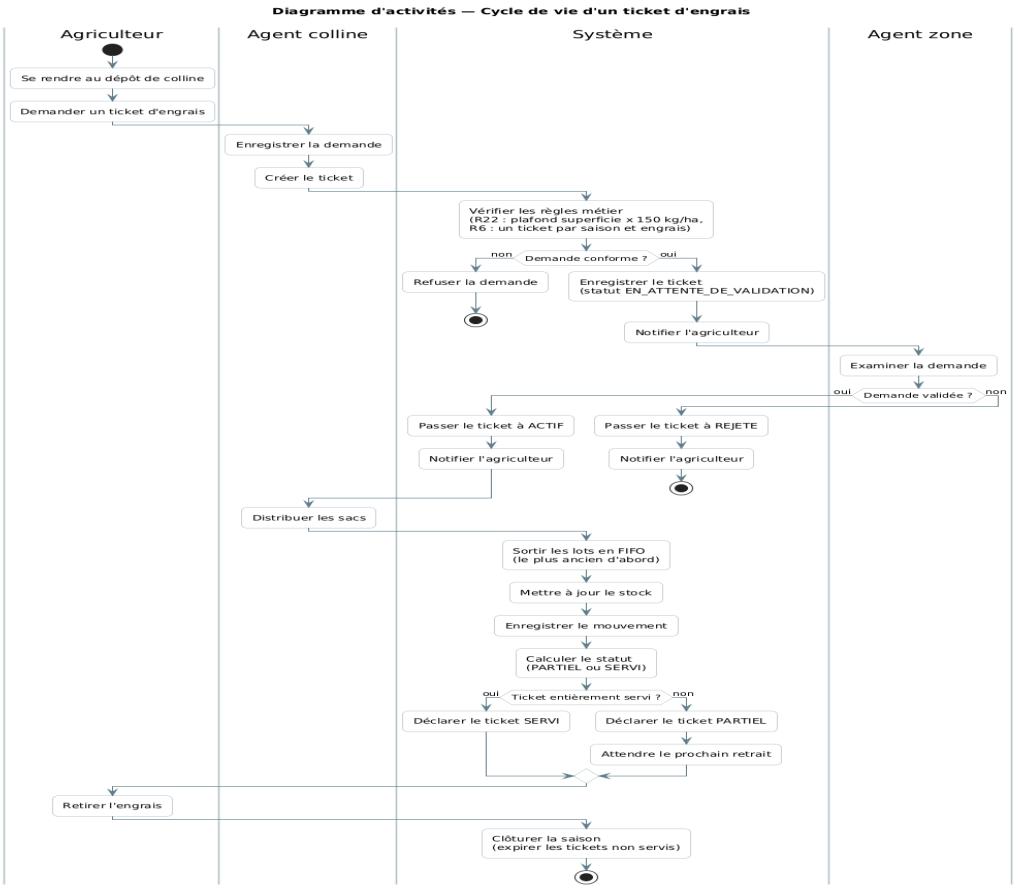


Figure III.2 — Diagramme d'activités : cycle de vie d'un ticket d'engrais

III.2 Conception du système

Cette section présente la conception du système : l'architecture d'ensemble, le modèle de données, le diagramme de séquence du traitement critique et la structure du projet. La conception s'appuie sur les besoins formalisés dans la section précédente et sur les contraintes techniques du contexte de déploiement.

III.2.1 L'architecture du système

L'architecture du système décrit l'organisation d'ensemble de la solution : les composants logiciels, leur rôle respectif et la manière dont ils communiquent (Figure III.3). Le système FOMI-Stock adopte une architecture en trois tiers, dans laquelle les clients web et mobile consomment une API REST unique qui s'appuie sur une base de données relationnelle. Cette organisation centralise la logique métier et les données, ce qui garantit que tous les acteurs —

administrateurs, superviseurs, agents de zone, agents de colline et agents logistiques — travaillent sur un même référentiel d'information.

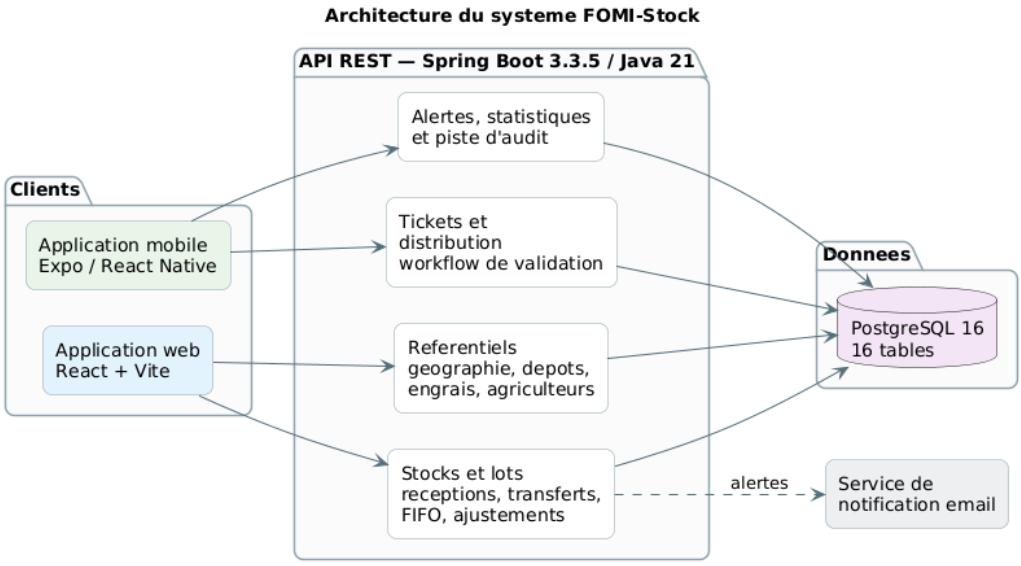


Figure III.3 — Architecture du système FOMI-Stock

Les clients constituent le premier tiers. L'application web, développée avec React et Vite, est destinée aux utilisateurs du siège et des dépôts provinciaux ; elle est configurée comme une application web progressive (PWA). L'application mobile, développée avec Expo et React Native, cible les agents de terrain qui travaillent au niveau des zones et des collines. Les deux clients échangent avec l'API au format JSON, sur le protocole HTTPS, et ne détiennent aucune donnée métier : toute la logique est portée par le serveur.

L'API REST, développée avec Java 21 et Spring Boot 3.3.5, constitue le deuxième tiers. Elle regroupe dix-sept contrôleurs exposant soixante-deux points d'entrée, répartis en quatre domaines fonctionnels : les référentiels (géographie, dépôts, types d'engrais, agriculteurs), les stocks et les lots (réceptions, transferts inter-dépôts, sorties, ajustements), les tickets et leur distribution (création, validation, distribution, annulation), et enfin les alertes, les statistiques et la piste d'audit. Chaque requête traverse le filtre d'authentification, qui vérifie le jeton et charge le profil de l'utilisateur, avant d'être traitée par le service métier compétent. Le service de notification complète l'ensemble en adressant les alertes par courrier électronique.

La base de données PostgreSQL 16 forme le troisième tiers. Elle stocke seize tables relationnelles dont le schéma est versionné par douze migrations Flyway. Les contraintes

d’intégrité — clés étrangères, énumérations nommées, contrôles de quantité et d’unicité — sont portées par la base elle-même, de sorte qu’aucune incohérence ne peut être enregistrée même en cas d’appel direct à l’API.

Le choix d’une API unique plutôt que de composants spécialisés répond à trois contraintes du contexte. D’abord, la taille de l’organisation ne justifie pas une infrastructure distribuée : un serveur unique réduit les coûts d’exploitation et de maintenance. Ensuite, la cohérence des stocks exige des transactions strictes entre les mouvements, les distributions et les tickets ; une base relationnelle unique garantit cette cohérence. Enfin, l’ouverture de l’API permet d’ajouter sans refonte des clients supplémentaires, par exemple un portail public de suivi des tickets ou une passerelle de paiement mobile.

III.2.2 Modèle de données

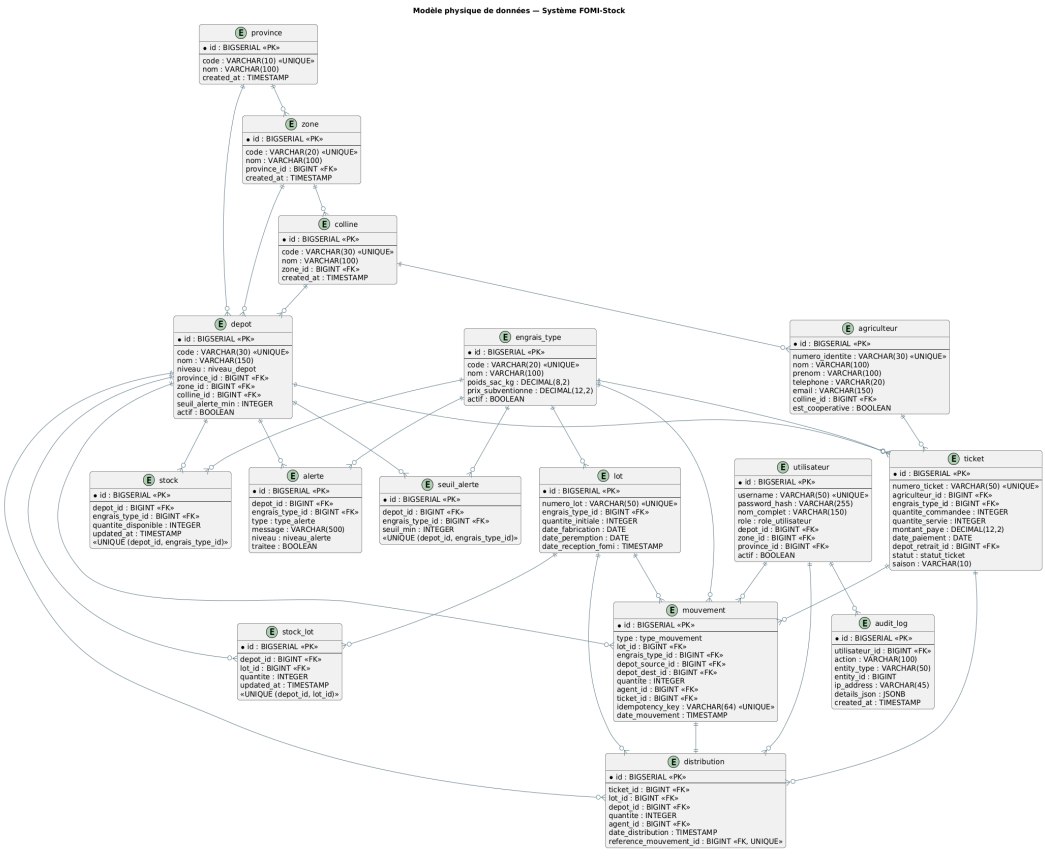


Figure III.4 — Diagramme Entité-Relation

Le modèle de données du système comprend seize tables principales, modélisant l’ensemble des entités métier et leurs relations. Le diagramme Entité-Relation (Figure III.4) présente ces

relations, tandis que le diagramme de classes (Figure III.5) en donne la traduction orientée objet. Le modèle suit les principes de normalisation de la 3ème forme (3NF), évitant les redondances de données et garantissant l'intégrité référentielle via des clés étrangères. Les relations sont principalement de type un-à-plusieurs (1:N) et plusieurs-à-plusieurs (M:N) avec tables de jointure.

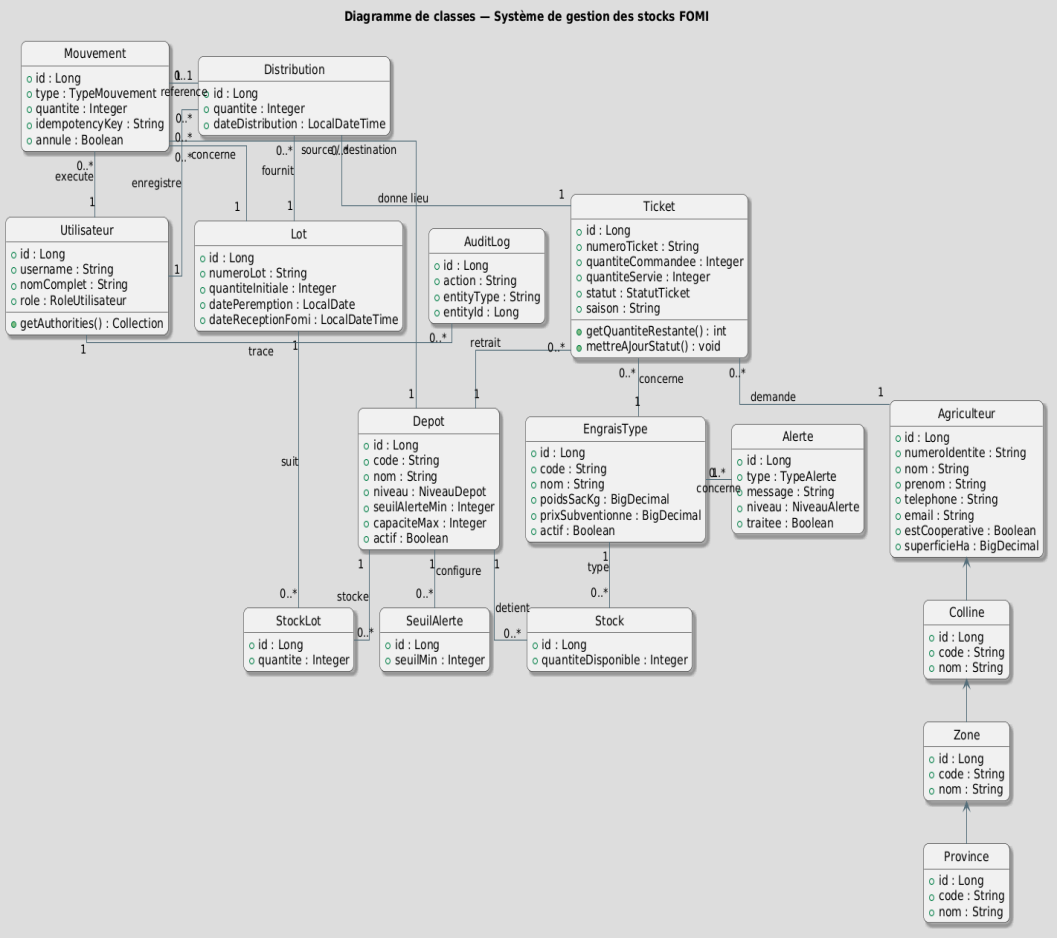


Figure III.5 — Diagramme de classes UML

Tableau III.5 — Description des tables de la base de données

Table	Description	Colonnes clés
province	Provinces du Burundi	id, code, nom, created_at
zone	Zones administratives rattachées aux provinces	id, code, nom, province_id, created_at
colline	Collines rattachées aux zones	id, code, nom, zone_id, created_at

Table	Description	Colonnes clés
depot	Dépôts de stockage (province, zone, colline)	id, code, nom, niveau, province_id, zone_id, colline_id, seuil_alerte_min, capacite_max, actif
engrais_type	Types d'engrais et prix subventionné	id, code, nom, poids_sac_kg, prix_subventionne, actif
lot	Lots d'engrais avec dates de fabrication et de péremption	id, numero_lot, engrais_type_id, quantite_initiale, date_fabrication, date_peremption, date_reception_fomi
stock	Stock agrégé par dépôt et type d'engrais	id, depot_id, engrais_type_id, quantite_disponible, updated_at
stock_lot	Stock détaillé par dépôt et lot	id, depot_id, lot_id, quantite, updated_at
agriculteur	Agriculteurs bénéficiaires	id, numero_identite, nom, prenom, telephone, colline_id, est_cooperative, superficie_ha
ticket	Tickets de distribution	id, numero_ticket, agriculteur_id, engrais_type_id, quantite_commandee, quantite_servie, montant_paye, date_paiement, depot_retrait_id, statut, saison
utilisateur	Utilisateurs et rôles	id, username, password_hash, nom_complet, role, depot_id, zone_id, province_id, actif
mouvement	Historique des mouvements de stock	id, type, lot_id, engrais_type_id, depot_source_id, depot_dest_id, quantite, agent_id, ticket_id, date_mouvement, annule
distribution	Détail des distributions effectuées	id, ticket_id, lot_id, depot_id, quantite, agent_id, date_distribution, reference_mouvement_id
alerte	Alertes automatiques	id, depot_id, engrais_type_id, type, message, niveau, traitee, created_at
audit_log	Piste d'audit	id, utilisateur_id, action, entity_type, entity_id, ip_address, details_json, created_at
seuil_alerte	Seuils d'alerte par dépôt et type d'engrais	id, depot_id, engrais_type_id, seuil_min, criticite

Le système exploite les énumérations nommées de PostgreSQL pour garantir l'intégrité des données. Les enums utilisés sont :

Tableau III.6 — Énumérations du système

Enum	Valeurs	Usage
NiveauDepot	PROVINCE, ZONE, COLLINE	Classement des dépôts par niveau hiérarchique
StatutTicket	EN_ATTENTE_DE_VALIDATION, ACTIF, REJETE, EXPIRE, PAYE, EN_ATTENTE, PARTIEL, SERVI, RECLAMATION, ANNULE	Workflow des tickets de distribution
TypeMouvement	RECEPTION_FOMI, TRANSFERT, DISTRIBUTION, AJUSTEMENT	Classification des mouvements de stock
TypeAlerte	RUPTURE, STOCK_BAS, DELAI_DEPASSE, DEMANDE_SUP_STOCK, PEREMPTION, INACTIVITE	Types d'alertes automatiques
NiveauAlerte	INFO, WARNING, CRITICAL	Sévérité des alertes
RoleUtilisateur	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE, AGENT_LOGISTIQUE	Profils utilisateurs du système

Tableau III.7 — Dictionnaire de données : colonnes, types SQL et contraintes

Table	Colonne	Type SQL / Enum	Contraintes / Description
utilisateur	username / password_hash	VARCHAR(50) / VARCHAR(255)	username UNIQUE NOT NULL ; hash BCrypt
utilisateur	nom_complet	VARCHAR(150)	NOT NULL
utilisateur	role	role_utilisateur (enum PG)	NOT NULL
utilisateur	depot_id / zone_id / province_id	BIGINT FK	NULL (ancrage géographique)
utilisateur	actif	BOOLEAN	DEFAULT TRUE ; soft

Table	Colonne	Type SQL / Enum	Contraintes / Description
			delete via supprime
depot	code	VARCHAR(30)	UNIQUE, NOT NULL
depot	niveau	niveau_depot (enum PG)	NOT NULL
depot	seuil_alerte_min	INTEGER	DEFAULT 100, NOT NULL
depot	capacite_max	INTEGER	NULL (V11)
ticket	numero_ticket	VARCHAR(50)	UNIQUE, NOT NULL
ticket	statut	statut_ticket (enum PG)	NOT NULL DEFAULT 'PAYE'
ticket	quantite_commandee / quantite_servie	INTEGER	NOT NULL / DEFAULT 0
ticket	agriculteur_id / depot_retrait_id	BIGINT FK	NOT NULL
ticket	montant_paye	DECIMAL(12,2)	NOT NULL
ticket	saison / superficie_ha	VARCHAR(10) / DECIMAL(8,2)	DEFAULT '2026A' / NULL
lot	numero_lot	VARCHAR(50)	UNIQUE, NOT NULL
lot	quantite_initiale	INTEGER	NOT NULL
lot	date_fabrication / date_peremption	DATE	NULL
lot	date_reception_fomi	TIMESTAMP	NOT NULL DEFAULT NOW() (base FIFO)
mouvement	type	type_mouvement (enum PG)	NOT NULL
mouvement	idempotency_key	VARCHAR(64)	UNIQUE (nullable)
mouvement	annule / motif_annulation / annule_par_id / reference_annulation_id	BOOLEAN / VARCHAR(500) / BIGINT FK / BIGINT FK	V11 (annulation)
stock / stock_lot	depot_id / engrais_type_id / lot_id	BIGINT FK	UNIQUE (depot_id, engrais_type_id) et (depot_id,

Table	Colonne	Type SQL / Enum	Contraintes / Description
			lot_id)
audit_log	action / entity_type / entity_id	VARCHAR / VARCHAR / BIGINT	NOT NULL / NOT NULL / NULL
audit_log	details_json	JSONB	Sans index GIN
alerte	depot_id / type / engrais_type_id	BIGINT FK	engrais_type_id NOT NULL (à assouplir pour INACTIVITE)
alerte	niveau	niveau_alerte (enum PG)	NOT NULL DEFAULT 'WARNING'

Le diagramme de classes présente les seize entités du système, leurs attributs principaux et les associations qui les relient : la hiérarchie géographique (province, zone, colline), les acteurs (agriculteur, utilisateur), les objets de stock (type d'engrais, lot, stock, stock par lot), les objets de flux (ticket, mouvement, distribution) et les objets de pilotage (alerte, seuil d'alerte, journal d'audit).

Le diagramme de déploiement (Figure III.6) montre la répartition des composants entre le client, le serveur API hébergé sur Railway et la base PostgreSQL 16, avec indication des conteneurs Docker et du pipeline CI/CD GitHub Actions.

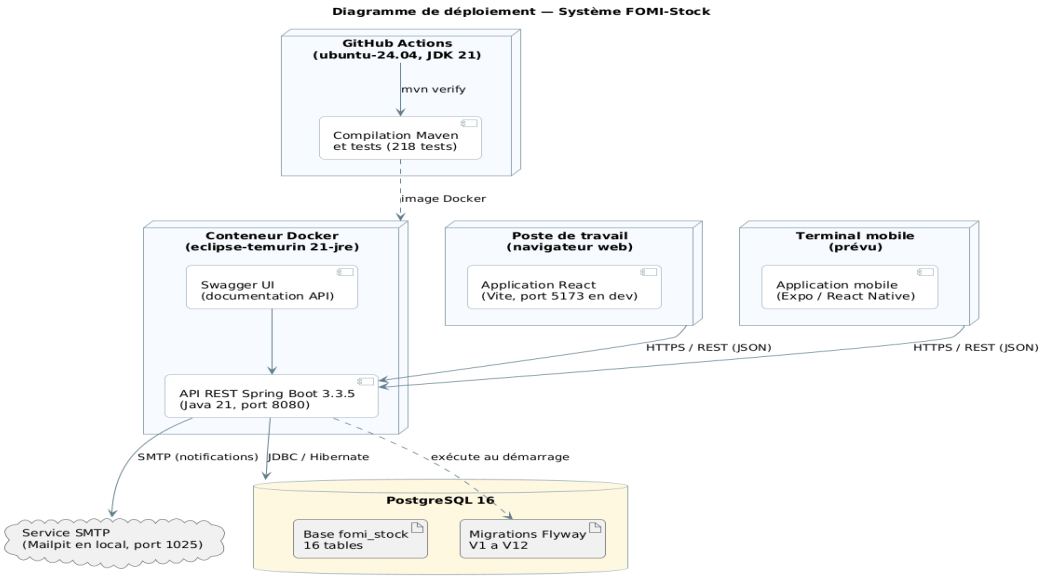


Figure III.6 — Diagramme de déploiement

III.2.3 Diagramme de séquence

Le diagramme de séquence modélise les interactions entre les composants du système lors d'une opération critique. Il illustre l'échange chronologique des messages entre l'utilisateur, l'API, les services métier et la base de données. Le scénario retenu est la création et la validation d'un ticket, qui constitue le point d'entrée du processus de distribution.

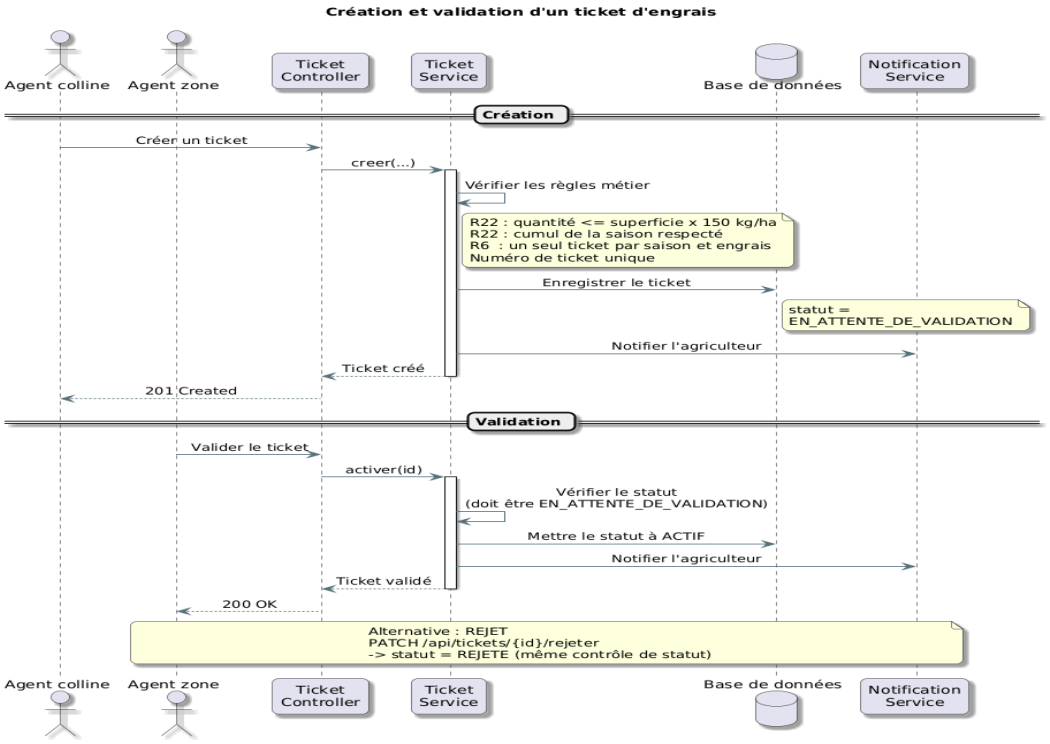


Figure III.7 — Diagramme de séquence : Création et validation de ticket

Le diagramme de séquence de la création et validation d'un ticket (Figure III.7) illustre le flux complet depuis la demande de l'agriculteur jusqu'à l'activation du ticket. L'agent de colline crée le ticket en spécifiant l'agriculteur, le type d'engrais et la quantité demandée. Le TicketController.recevoir() valide le format de la requête via les annotations Bean Validation, puis transmet au TicketService.creer(). Ce dernier vérifie automatiquement : la conformité avec le quota de superficie (R2 : superficie × 150 kg/ha), l'unicité par saison (R6 : pas de doublon même saison + même type), la disponibilité du stock (R3 : vérification seuil) et l'existence de l'agriculteur. Si toutes les vérifications passent, le ticket est créé avec le statut EN_ATTENTE_DE_VALIDATION. Un responsable peut ensuite valider le ticket (passage à ACTIF) ou le rejeter (passage à REJETE avec motif).

III.3 Réalisation, tests et évaluation

Cette section présente la réalisation du système, les tests qui l'ont validé et l'évaluation de ses résultats. Elle décrit d'abord la structure du projet et les principaux mécanismes implémentés, puis la stratégie de test, le déploiement et les enseignements tirés de la mise en œuvre.

III.3.1 Structure du projet

Le code source est organisé en packages, chacun regroupant les classes ayant une responsabilité commune : les contrôleurs exposent l'API, les services portent la logique métier, les repositories assurent l'accès aux données et les entités décrivent le modèle persistant. Cette organisation en packages suit les conventions du framework Spring Boot et permet une navigation facile dans le code source.

Package	Contenu	Nombre de classes
controller	Contrôleurs REST exposant les endpoints API	17 classes
service	Services contenant la logique métier	14 classes
repository	Repositories Spring Data JPA	16 interfaces
model/entity	Entités JPA (tables de la base)	16 classes
model/enums	Énumérations Java	6 enums
dto	Objets de transfert de données	34 classes
security	Filtre JWT, provider auth, utils sécurité	4 classes
config	Configuration Spring (CORS, Swagger, scheduling)	1 classe
exception	Exceptions métier et gestion centralisée	3 classes

Cette organisation permet une séparation claire des responsabilités et facilite la maintenabilité du code. Les dépendances entre packages restent unidirectionnelles : un contrôleur invoque un service, le service s'appuie sur un ou plusieurs repositories, et les repositories manipulent les entités. Ce sens unique de dépendance réduit les couplages et rend les services testables par simulation (Mockito). Les exceptions sont gérées de manière centralisée via @ControllerAdvice et @ExceptionHandler, ce qui garantit des réponses d'erreur uniformes et structurées pour tous les endpoints.

III.3.2 Système d’alertes

Le système d'alertes identifie et signale automatiquement les situations critiques nécessitant une intervention humaine. Six types d'alertes sont implémentés, chacun avec un déclencheur spécifique et un niveau de sévérité hiérarchique.

Tableau III.8 — Types d'alertes et déclencheurs

Type	Sévérité	Condition de déclenchement	Fréquence
RUPTURE	CRITICAL	Stock = 0 pour un type dans un dépôt	À chaque mouvement de stock
STOCK_BAS	WARNING	Stock < seuil_min configuré pour type/dépôt	À chaque mouvement de stock
DELAI_DEPASSE	WARNING	Ticket non servi dont la date de paiement dépasse 7 jours	Cron 06h00 quotidien
DEMANDE_SUP_STOCK	WARNING	Demande non servie > stock disponible	À chaque mouvement de stock (verifierDepot)
PEREMPTION	CRITICAL	Lot déjà périmé avec stock > 0	À chaque mouvement de stock (verifierDepot)
INACTIVITE	WARNING	Aucun mouvement sur un dépôt actif pendant 7 jours	Cron 06h30 quotidien

Le mécanisme de dédoublement est essentiel pour éviter la multiplication d'alertes identiques. La méthode AlerteService.creerSiAbsente() vérifie l'existence d'une alerte non traitée de même type, même dépôt et même type d'engrais avant d'en créer une nouvelle. Si une alerte similaire existe déjà, aucune nouvelle alerte n'est créée.

Deux tâches planifiées via l'annotation @Scheduled complètent la détection en temps réel : la première s'exécute à 06h00 chaque jour pour vérifier les tickets non servis dont la date de paiement dépasse 7 jours (alertes DELAI_DEPASSE) ; la seconde s'exécute à 06h30 pour identifier les dépôts actifs sans aucun mouvement de stock depuis 7 jours (alertes

INACTIVITE). Les alertes de rupture, de stock bas, de péremption et de demande supérieure au stock sont, elles, générées par AlerteService.verifierDepot() après chaque mouvement de stock.

III.3.3 Workflow de validation des tickets

Le workflow de validation des tickets implémente un processus en deux étapes avec vérifications automatiques à la création. Lors de la création du ticket (TicketService.creer()), le système vérifie plusieurs règles métier : le quota de superficie (R2 : la quantité demandée ne doit pas dépasser superficie × 150 kg/ha), le cumul des tickets de la même saison, l'unicité par saison et type d'engrais (R6), ainsi que l'existence de l'agriculteur, du type d'engrais et du dépôt de retrait. La disponibilité du stock n'est pas contrôlée à la création ; elle est vérifiée au moment de la distribution.

Lors de la validation par le responsable (TicketService.activer()), le système vérifie que le ticket est bien en EN_ATTENTE_DE_VALIDATION puis le passe à ACTIF. Aucune re-vérification du stock ou des quotas n'est effectuée à cette étape. Le responsable peut également rejeter le ticket via TicketService.rejeter() avec un motif ; le statut passe à REJETE. Une fois le ticket ACTIF, il peut être traité par un agent lors de la distribution via DistributionService.distribuer().

III.3.4 Annulation des mouvements

Le mécanisme d'annulation des mouvements est une fonctionnalité critique pour la correction des erreurs et la gestion des litiges. Deux types d'annulation sont supportés, implémentés dans MouvementService.annuler() : l'annulation d'un transfert et celle d'une distribution.

Annulation d'une distribution : lorsqu'un mouvement de type DISTRIBUTION est annulé, le système décrémente la quantite_servie du ticket associé, met à jour son statut et supprime l'enregistrement de distribution correspondant. En revanche, le stock du dépôt n'est pas réincrémenté, ce qui peut créer un écart entre le stock physique et le stock logique. Un mouvement de type AJUSTEMENT est créé avec une référence vers le mouvement annulé.

Annulation d'un transfert : lorsqu'un mouvement de type TRANSFERT est annulé, le stock est restauré au dépôt source (incrémenté) et retiré du dépôt destination (décrémenté). Cette

opération atomique garantit la cohérence des stocks entre les deux dépôts. Un mouvement de type AJUSTEMENT est créé avec la référence du transfert original.

Dans les deux cas, des vérifications sont effectuées avant l'annulation : le mouvement ne doit pas déjà être annulé (R7 : anti-double annulation), une réception FOMI ou un ajustement ne peut pas être annulé, et un log d'audit est généré pour tracer l'opération de correction. L'annulation d'un transfert vérifie également la disponibilité du stock au dépôt destination.

III.3.5 Statistiques et tableaux de bord

Le système expose plusieurs endpoints de reporting via StatsController, permettant aux responsables et administrateurs de visualiser l'état du système et de prendre des décisions éclairées. Les statistiques sont calculées en temps réel à partir des données de la base, sans cache, pour garantir leur fraîcheur.

Taux de service par province/zone : calculé comme le ratio entre le nombre de tickets SERVI et le nombre total de tickets créés (SERVI + PARTIEL + ACTIF + REJETE + ANNULE), ventilé par province ou zone. Cet indicateur mesure l'efficacité du système de distribution et permet d'identifier les zones sous-performantes. Le calcul est effectué via une requête JPQL avec GROUP BY province_id/zone_id.

Stocks par province : agrégation des stocks de tous les dépôts d'une province, ventilée par type d'engrais. Les données sont présentées sous forme de tableau avec les colonnes province, type d'engrais, stock_total et statut (OK/BAS/CRITIQUE). Cette vue permet d'identifier rapidement les provinces sous-approvisionnées nécessitant un transfert de stock.

Ruptures en cours : liste des dépôts ayant un stock nul pour un ou plusieurs types d'engrais, avec l'historique des alertes de rupture associées. Cette information est critique pour les opérations de distribution car elle identifie les points bloquants de la chaîne.

Tickets non servis par colline : liste des tickets en état ACTIF ou PARTIEL dont la distribution n'est pas complète, ventilée par colline avec le nombre de tickets et la quantité totale en attente. Cet indicateur permet de prioriser les distributions et d'identifier les zones où les agriculteurs sont les plus en attente.

Mouvements par agent : historique des opérations effectuées par chaque agent, avec comptage par type de mouvement (RECEPTION_FOMI, TRANSFERT, DISTRIBUTION,

AJUSTEMENT). Utile pour le suivi de productivité, la détection d'anomalies et la préparation des audits internes.

III.3.6 Piste d'audit

Le mécanisme d'audit trail est implémenté via la classe AuditLogService, appelée explicitement dans les services lors des opérations critiques (création, validation, distribution, annulation, réception, transfert). Il n'existe pas d'aspect AOP automatique de type AuditLogAspect ni d'annotation @Auditable : l'appel à AuditLogService.log() est donc manuel et doit être maintenu par les développeurs.

Champ	Type	Description
action	VARCHAR	Type d'opération : TICKET_CREATE, TICKET_ACTIVATE, DISTRIBUTION_CREATE, MOUVEMENT_CANCEL, etc.
entity_type	VARCHAR	Type d'entité concernée : Ticket, Agriculteur, Lot, Mouvement, etc.
entity_id	BIGINT	Identifiant de l'entité modifiée
utilisateur_id	BIGINT	Identifiant de l'utilisateur ayant effectué l'opération
ip_address	VARCHAR	Adresse IP de la requête (RemoteAddr)
details_json	JSONB	Détails de l'opération (ancien/nouveau valeur, paramètres)
created_at	TIMESTAMP	Horodatage de l'opération

Les logs d'audit sont stockés dans la table audit_log avec des index optimisés pour les requêtes de consultation courantes : un index sur entity_type + entity_id pour les recherches par entité, un index sur utilisateur_id pour les recherches par agent et un index sur created_at pour le tri chronologique. L'endpoint de consultation (AuditLogController) offre une pagination plafonnée à 200 éléments par page et des filtres par action, type d'entité, identifiant d'entité et utilisateur. Il ne propose pas de filtre par plage de dates. La colonne details_json est de type JSONB, mais aucun index GIN n'y est associé.

III.3.7 Tests et déploiement

Cette section présente la stratégie de test adoptée pour le projet, les différents types de tests implémentés, l'infrastructure de CI/CD et les résultats obtenus. La qualité des tests est un élément central du projet, garantissant la fiabilité du système et la confiance dans ses implémentations.

III.3.7.1 Stratégie de test

La stratégie de test du projet repose principalement sur des tests d'intégration. Les 215 tests d'intégration sont exécutés avec une instance PostgreSQL réelle lancée via Testcontainers et vérifient le comportement de l'API de bout en bout, de l'interface HTTP à la base de données. Ils sont complétés par 3 tests unitaires de configuration. Le projet ne comporte donc pas de pyramide de tests classique majoritairement unitaire.

Les tests d'intégration sont exécutés avec une instance PostgreSQL réelle lancée via Testcontainers. Ils vérifient le fonctionnement correct de l'ensemble du système, de l'API à la base de données, en incluant les transactions, les contraintes d'intégrité référentielle et les index de performance. L'utilisation de Testcontainers garantit que les tests s'exécutent dans un environnement identique à la production, éliminant les bugs liés aux différences comportementales entre une base en mémoire et PostgreSQL.

III.3.7.2 Tests d'intégration

Les tests d'intégration sont organisés dans une classe abstraite BaseIntegrationTest qui fournit les méthodes utilitaires suivantes : setupJwt(role, depotId) configure un token JWT valide pour un rôle et un périmètre spécifiques ; apiGet(uri) effectue une requête GET authentifiée ; apiPost(uri, body) effectue une requête POST avec un corps JSON ; apiPut(uri, body) et apiDelete(uri) pour les requêtes PUT et DELETE. Cette base commune assure la cohérence et la réutilisabilité des tests.

Tableau III.9 — Récapitulatif des tests d'intégration

Suite de tests	Nombre	Domaine couvert
ValidationTicketIntegrationTest	27 tests	Workflow complet : création, activation, rejet, unicité, quota

Suite de tests	Nombre	Domaine couvert
SecuriteAccesIntegrationTest	18 tests	Contrôle d'accès : rôles, périmètre, accès refusé, jeton invalide
DistributionIntegrationTest	17 tests	Distribution, contrôle du restant, mise à jour du ticket
AuthIntegrationTest	17 tests	Authentification, JWT, comptes désactivés
AgriculteurIntegrationTest	15 tests	CRUD, recherche, téléphone unique
TicketIntegrationTest	14 tests	CRUD tickets, filtres, pagination
DepotIntegrationTest	14 tests	CRUD dépôts, contraintes de niveau, stocks
MouvementIntegrationTest	14 tests	Réception, transfert, annulation, ajustement
UtilisateurIntegrationTest	11 tests	CRUD utilisateurs, rôles, soft delete
ApiIntegrationTest	9 tests	Endpoints transverses et gestion des erreurs
AlerteIntegrationTest	8 tests	Déclenchement, dédoublement, traitement
GeoIntegrationTest	7 tests	Provinces, zones, collines
StockIntegrationTest	6 tests	Consultation, disponibilité, seuils
LotIntegrationTest	6 tests	Création, recherche, péremption
DistributionFifoIntegrationTest	5 tests	FIFO multi-lots, partiel, complet
AuditLogIntegrationTest	4 tests	Enregistrement, consultation, filtres, pagination
StatsIntegrationTest	4 tests	Taux de service, stocks par province, ruptures
GlobalExceptionHandlerIntegrationTest	4 tests	Réponses d'erreur uniformes
AlerteDelaiIntegrationTest	3 tests	Alertes de délai dépassé
StockConcurrenceIntegrationTest	1 test	Pessimistic locking, accès concurrent
Total	215 tests	22 classes de tests d'intégration PostgreSQL

III.3.7.3 Intégration et déploiement continus

Le pipeline d'intégration continue et de déploiement continu (CI/CD) est implémenté avec GitHub Actions. La configuration (.github/workflows/ci.yml) définit les étapes suivantes : checkout du code, configuration de Java 21, exécution des tests (mvn test), construction du

JAR (mvn package), et déploiement conditionnel. À chaque push ou pull request, le pipeline exécute automatiquement la compilation et l'ensemble de la suite de tests. En cas d'échec d'un test, le merge de la branche est bloqué, garantissant la qualité du code dans la branche principale (main).

Le déploiement en production est assuré via Docker et Railway. L'application est conteneorisée avec un Dockerfile optimisé multi-stage (construction avec Maven, exécution avec JRE léger). Les migrations de base de données sont exécutées automatiquement au démarrage du conteneur via Flyway (spring.flyway.enabled=true). Cette approche de déploiement continu permet de livrer les nouvelles fonctionnalités et les corrections de bugs de manière rapide et fiable, avec un temps de déploiement moyen de moins de 5 minutes.

III.3.7.4 Résultats

La suite de tests complète compte 218 tests automatisés, tous en succès (218/218). La couverture de code n'a pas été mesurée avec un outil dédié ; elle n'est donc pas chiffrée dans ce rapport. Les tests couvrent l'ensemble des besoins fonctionnels critiques (F1-F25), des règles métier (R1-R22) et des cas d'utilisation (UC-01 à UC-26).

Le temps d'exécution total de la suite est d'environ 3 minutes, dont une part importante correspond au démarrage de l'instance Docker PostgreSQL utilisée par Testcontainers. Les 215 tests d'intégration représentent l'essentiel de la durée, les 3 tests unitaires étant quasi instantanés.

III.3.8 Évaluation des résultats

Cette section évalue le système réalisé par rapport aux objectifs fixés, en termes de conformité au cahier des charges, de performance, de maintenabilité et de sécurité. L'évaluation est structurée de manière à couvrir l'ensemble des dimensions de qualité du système et à fournir un bilan objectif des réalisations et des limites du projet.

III.3.8.1 Conformité au cahier des charges

Le tableau ci-dessous présente la conformité du système réalisé par rapport aux 25 besoins fonctionnels du cahier des charges. Chaque besoin a été implémenté dans les composants appropriés du système et validé par des tests d’intégration spécifiques.

ID	Besoins	Statut	Référence code
F1	Authentification JWT	Implémenté	SecurityConfig, AuthController
F2	Types d'engrais CRUD	Implémenté	EngraisTypeController
F3	Géographie hiérarchique	Implémenté	GeoController
F4	Dépôts CRUD	Implémenté	DepotController, DepotService
F5	Agriculteurs CRUD	Implémenté	AgriculteurController
F6	Tickets distribution	Implémenté	TicketController, TicketService
F7	Réception FOMI	Implémenté	MouvementService.reception()
F8	Transferts inter-dépôts	Implémenté	MouvementService.transfert()
F9	Distribution FIFO	Implémenté	DistributionService.distribuer()
F10	Gestion des lots	Implémenté	LotController
F11	Suivi stocks	Implémenté	StockController, StockService
F12	Alertes automatiques	Implémenté	AlerteService, @Scheduled
F13	Annulation mouvements	Implémenté	MouvementService.annuler()
F14	Validation tickets	Implémenté	TicketService.activer(), rejeter()
F15	Statistiques	Implémenté	StatsController, StatsService
F16	Export CSV	Implémenté	ExportController
F17	Audit trail	Implémenté	AuditLogService, AuditLogController
F18	Seuils alerte	Implémenté	SeuilAlerteController
F19	Consultation publique	Implémenté	PublicTicketController
F20	Recherche multicritère	Implémenté	AgriculteurController.recherche(), LotController.recherche()
F21	Tickets non servis	Implémenté	StatsController (/tickets-non-servis-par-colline)
F22	Taux service	Implémenté	StatsController (/taux-service, /taux-service-par-province)
F23	Mouvements par agent	Implémenté	StatsController (/mouvements-par-

ID	Besoins	Statut	Référence code
			agent/{agentId})
F24	Gestion utilisateurs	Implémenté	UtilisateurController, UtilisateurService
F25	Intégrité	Implémenté	Contraintes SQL, @Transactional, MouvementService.annuler()

L'analyse de conformité révèle un taux d'implémentation de 100 % pour l'ensemble des 25 besoins fonctionnels. Chaque besoin a été implémenté et validé par des tests d'intégration correspondants, garantissant non seulement la présence du code mais aussi son fonctionnement correct dans les conditions réelles d'utilisation.

III.3.8.2 Performance

Les performances du système ont été évaluées en termes de temps de réponse et d'efficacité des requêtes. Les principaux optimisations implémentées incluent :

Pagination : les endpoints de liste (tickets, agriculteurs, mouvements, audit) utilisent une pagination côté serveur via Spring Data. Les contrôleurs plafonnent la taille de page à 200 éléments. Certains référentiels (types d'engrais, provinces, zones, collines, dépôts, lots, stocks) renvoient l'ensemble des éléments sans pagination.

Index de base de données : des index B-tree ont été créés sur les clés étrangères et les colonnes de filtrage fréquentes : lot_id, depot_source_id, depot_dest_id, ticket_id, statut, date_mouvement, utilisateur_id. Des index composites couvrent (depot_retrait_id, statut) pour les tickets, (lot_id, date_mouvement) pour les mouvements, (entity_type, entity_id) et (utilisateur_id, created_at) pour l'audit, ainsi que (depot_id, traitee) pour les alertes. La contrainte UNIQUE (depot_id, engrais_type_id) de la table stock joue également le rôle d'index composite. Aucun index GIN n'a été créé sur details_json.

Pessimistic locking : les opérations critiques de modification de stock utilisent SELECT FOR UPDATE pour éviter les conditions de course (race conditions) en présence de transactions concurrentes. Ce mécanisme, bien que réduisant légèrement le débit en cas de forte concurrence, garantit l'intégrité des données et évite les pertes de stock inexpliquées.

Requêtes JPQL optimisées : les agrégations des tableaux de bord sont effectuées côté serveur SQL (SUM, COUNT, GROUP BY) plutôt qu'en mémoire Java, réduisant la consommation de mémoire et le temps de traitement. Le service de statistiques ne recourt toutefois pas systématiquement à JOIN FETCH ni à DISTINCT.

III.3.8.3 Maintenabilité

La maintenabilité du système est un critère essentiel pour un projet destiné à être maintenu et évolué par des équipes internes à FOMI. Plusieurs facteurs y contribuent : la séparation nette des responsabilités entre les contrôleurs, les services et les repositories permet de localiser rapidement les modifications nécessaires ; la documentation OpenAPI offre une vue complète et interactive de l'API, ce qui réduit le temps d'appropriation par un nouveau développeur ; les 218 tests automatisés garantissent que les modifications n'introduisent pas de régressions ; la gestion des migrations Flyway assure la traçabilité des évolutions du schéma ; enfin, les conventions de nommage cohérentes facilitent la navigation dans le code source.

III.3.8.4 Sécurité

Le système intègre un ensemble complet de mécanismes de sécurité conformes aux meilleures pratiques de l'industrie :

— Cinq profils d'utilisateurs (ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE, AGENT_LOGISTIQUE) délimitent les permissions de chaque type d'utilisateur selon le principe du moindre privilège.

— Authentification stateless via JWT empêchant les attaques de type session hijacking et facilitant la montée en charge horizontale.

— Hashage BCrypt des mots de passe (cost factor 10) résistant aux attaques par dictionnaire et par force brute. Les mots de passe ne sont jamais stockés en clair.

— Contrôles de périmètre géographique dans les services et contrôleurs : un agent de colline ne peut accéder qu'aux données de son dépôt et un agent de zone uniquement aux dépôts de sa zone. Il n'existe pas d'aspect AOP dédié (SecurityScopeAspect) ; ces contrôles sont explicites dans le code.

— Validation de toutes les entrées utilisateur via les annotations Bean Validation (@NotNull, @Min, @Size, @Pattern) empêchant l'injection de données malveillantes.

— Les en-têtes de sécurité HTTP (Content-Security-Policy, X-Content-Type-Options, X-Frame-Options) ne sont pas configurés explicitement dans l'API actuelle ; leur ajout constitue une recommandation.

Deux clients ont été développés pour consommer l'API. L'application web, développée avec React et Vite, comporte douze pages couvrant le tableau de bord, les dépôts, les tickets, les distributions, les agriculteurs, les mouvements, les lots, les alertes, les utilisateurs, les journaux d'audit et le suivi public des tickets. Elle est configurée comme une application web progressive (PWA) et déployée via GitHub Pages.

L'application mobile, développée avec Expo et React Native, comporte treize écrans et reprend les principales fonctionnalités de terrain : tableau de bord, tickets, stocks, alertes, agriculteurs, distributions, mouvements, lots, utilisateurs, audit et suivi public. Elle utilise React Navigation et AsyncStorage pour la session.

Ces deux clients constituent des prototypes fonctionnels : ils illustrent l'intégration de l'API mais ne bénéficient pas encore de tests automatisés et présentent quelques écarts de contrat avec certains endpoints. Les tests présentés dans ce mémoire portent donc exclusivement sur l'API.

CONCLUSION GÉNÉRALE

Ce travail avait pour objectif de concevoir et de réaliser un système d'information centralisé pour la gestion des stocks d'engrais de la société FOMI au Burundi. Partant du constat d'une gestion manuelle, dispersée et peu traçable, nous avons analysé l'existant, formalisé les besoins et conçu une solution logicielle complète.

Les résultats obtenus sont les suivants : une API REST documentée exposant soixante-deux endpoints, couvrant les référentiels, les réceptions, les transferts, la distribution FIFO, les tickets et leur workflow, les alertes, les statistiques, les exports CSV et la piste d'audit ; un modèle de données relationnel de seize tables géré par douze migrations Flyway ; un mécanisme de sécurité JWT avec cinq rôles et un hachage BCrypt ; un système d'alertes automatiques ; et une suite de deux cent sept tests automatisés, dont deux cent quatre tests d'intégration sur PostgreSQL réel et trois tests unitaires.

Sur le plan technique, le système repose sur Java 21, Spring Boot 3.3.5, PostgreSQL 16, Flyway, Spring Security, Spring Data JPA, SpringDoc OpenAPI et Testcontainers. Deux clients prototypes, l'un web (React) et l'autre mobile (Expo/React Native), ont également été développés pour illustrer l'utilisation de l'API.

Les hypothèses de recherche sont globalement confirmées : la centralisation des données, la traçabilité des mouvements, le contrôle d'accès par rôles et les alertes automatiques apportent une réponse pertinente aux dysfonctionnements observés. Le travail comporte néanmoins des limites, notamment l'absence de notifications SMS, de paiement mobile, de mode hors-ligne et de tests automatisés pour les clients. Ces limites ouvrent la voie à plusieurs perspectives d'amélioration présentées ci-après.

CRITIQUES, SUGGESTIONS ET RECOMMANDATIONS

1. Critiques

Malgré les résultats obtenus, plusieurs critiques peuvent être formulées à l'égard du système et de sa mise en œuvre.

1.1 Critiques techniques

— L'algorithme FIFO est ordonné par date de réception FOMI et non par date de péremption ; de plus, le chemin avec lot imposé ne vérifie pas la péremption.

— L'annulation d'une distribution ne restaure pas le stock du dépôt, ce qui peut entraîner un écart entre le stock physique et le stock logique.

— L'ordre des règles de sécurité permet à tout utilisateur authentifié d'annuler un mouvement, alors que cette opération devrait être réservée aux profils habilités.

— L'alerte INACTIVITE peut être créée sans type d'engrais, alors que le schéma impose une contrainte NOT NULL.

— L'expiration automatique des tickets existe dans le code mais n'est reliée à aucune tâche planifiée ni à aucun endpoint.

— Les clients web et mobile présentent des écarts de contrat avec l'API (champs obligatoires manquants, endpoints inexistants) et ne disposent pas de tests automatisés.

1.2 Critiques organisationnelles

— La qualité des données historiques est variable, ce qui limite la comparaison avant/après.

— La couverture réseau et l'équipement informatique des agents de terrain restent insuffisants dans certaines zones.

— L'appropriation du système par les agents nécessite une formation et un accompagnement dans la durée.

2. Suggestions

Pour améliorer le système, nous suggérons de :

— Corriger l'ordonnancement FIFO pour retenir la date de péremption et contrôler systématiquement la péremption, y compris pour les lots imposés.

— Restaurer le stock lors de l'annulation d'une distribution et recalculer correctement le statut du ticket.

— Réordonner les règles de sécurité afin que l'annulation des mouvements soit réservée aux profils autorisés.

— Rendre le type d'engrais facultatif dans la table des alertes ou adapter la contrainte pour les alertes d'inactivité.

— Planifier l'expiration automatique des tickets et ajouter les tests correspondants.

— Aligner les contrats des clients web et mobile sur l'API et ajouter des tests de non-régression.

— Compléter la couverture de tests unitaires et mettre en place un rapport de couverture.

3. Recommandations

À l'endroit de FOMI, nous recommandons un déploiement progressif du système, la désignation d'un administrateur fonctionnel, la formation des agents, la mise en place de sauvegardes automatiques et l'instauration d'indicateurs de suivi (taux de service, ruptures, péremptions).

À l'endroit des autorités publiques et du PNSEB, nous recommandons l'intégration du système au circuit officiel de distribution, la sécurisation des données et des accès, ainsi que la généralisation progressive à l'ensemble des provinces.

Pour les travaux futurs, plusieurs perspectives sont envisageables : notifications SMS, intégration du paiement mobile (Lumicash, EcoCash), mode hors-ligne avec synchronisation, tableaux de bord en temps réel (WebSocket), et modèles prédictifs d'estimation de la demande.

RÉFÉRENCES BIBLIOGRAPHIQUES

[1] Banque Mondiale. "Burundi Overview — Country Diagnostic." World Bank Group, Washington D.C., 2024.

[2] FAO. "Fertilizer Use by Crop in the United Republic of Burundi." Food and Agriculture Organization, Rome, 2022.

[3] Gouvernement du Burundi. "Décret n° 100-119 du 20 mai 2013 portant création du Fonds Commun pour les Fertilisants et Amendements (FCFA)." Journal Officiel de la République du Burundi, 2013.

[4] IWACU. "Distribution des engrais FOMI : Irrégularités dénoncées à Cibitoke." Investigation journalistique, février 2026.

[5] PostgreSQL Global Development Group. "PostgreSQL 16 Reference Manual." 2024.

[6] Fowler, M. "Patterns of Enterprise Application Architecture." Addison-Wesley Professional, Boston, 2002.

[7] Newman, S. "Building Microservices: Designing Fine-Grained Systems." O'Reilly Media, 2nd edition, 2021.

[8] FOMI SA. "Rapport annuel d'activités 2024." Fertilisants Organo-Minéraux Industries, Maramvya, 2024.

[9] Walls, C. "Spring Boot in Action." Manning Publications, Shelter Island, 2016.

[10] ISO 9001:2015. "Systèmes de management de la qualité — Exigences." Organisation Internationale de Normalisation, Genève, 2015.

[11] ISO 14001:2015. "Systèmes de management environnemental — Exigences et lignes directrices." ISO, 2015.

[12] ISO 45001:2018. "Systèmes de management de la santé et de la sécurité au travail." ISO, 2018.

[13] Tanzania Fertilizer Regulatory Authority. "Annual Report 2023 — Fertilizer Distribution and Regulation." TFRA, Dar es Salaam, 2023.

[14] Fielding, R.T. "Architectural Styles and the Design of Network-based Software Architectures." Doctoral dissertation, University of California, Irvine, 2000.

[15] Jacob, D. "Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions." Addison-Wesley, 2003.

[16] Sandhu, R.S., Coyne, E.J., Feinstein, H.L., and Youman, C.E. "Role-Based Access Control Models." IEEE Computer, 29(2):38-47, 1996.

[17] ISABU. "Rapport d'essais agronomiques — Efficacité des engrais organo-minéraux FOMI." Institut des Sciences Agronomiques du Burundi, 2023.

[18] Harris, F.W. "How Many Parts to Make at Once." Factory, The Magazine of Management, 10(2):135-136, 1913.

[19] Richardson, L. "Microservices Patterns: With Examples in Java." Manning Publications, 2018.

WEBOGRAPHIE

[20] SOS Médias Burundi. « Fertilizer Scandal in Burundi: Billions Claimed, Unclear Responsibilities. » [en ligne], https://www.sosmediasburundi.org/en/2026/01/03/fertilizer-scandal-in-burundi-billions-claimed-unclear-responsibilities-and-farmers-held-hostage/, consulté le 7 novembre 2026.

[21] SOS Médias Burundi. « Burunga : la crise des engrais tourne au cauchemar, les agriculteurs craignent une catastrophe alimentaire. » [en ligne], https://www.sosmediasburundi.org/en/2026/06/24/burunga-the-fertilizer-crisis-turns-into-a-nightmare-farmers-fear-a-food-catastrophe/, consulté le 7 novembre 2026.

[22] Andika Magazine. « Tensions entre le ministre des Finances et la FOMI : 81,6 milliards en jeu dans le différend sur les engrais. » [en ligne], https://andikamagazine.net/tensions-between-burundis-finance-minister-and-fomi-bif-81-6-billion-at-stake-in-fertilizer-dispute/, consulté le 7 novembre 2026.

[23] IWACU Burundi. « Engrais FOMI : des irrégularités dans la distribution persistent. » [en ligne], https://www.iwacu-burundi.org/engrais-fomi-des-irregularites-dans-la-distribution-persistent/, consulté le 7 novembre 2026.

[24] CGIAR. « Rwanda's Digital Revolution in Agriculture: Challenges and Progress in Smart Fertilizer Management. » [en ligne], https://www.cgiar.org/news-events/news/rwandas-digital-revolution-in-agriculture-challenges-and-progress-in-smart-fertilizer-management, consulté le 7 novembre 2026.

[25] CGIAR CGSpace. « Smart Nkunganire System — évaluation de la plateforme de subvention des intrants au Rwanda. » [en ligne], https://cgspace.cgiar.org/items/136fbcf4-94c8-4411-a9e2-803f794bd6e6, consulté le 7 novembre 2026.

[26] CGIAR CGSpace. « Digital tools for fertilizer subsidy management in East Africa. » [en ligne], https://cgspace.cgiar.org/items/92ed3212-3eed-4b59-bb84-8ee93b3a4255, consulté le 7 novembre 2026.

[27] Safaricom. « Ministry of Agriculture selects Safaricom to streamline the subsidy voucher programme. » [en ligne], https://www.safaricom.co.ke/media-center-landing/press-releases/ministry-of-agriculture-selects-safaricom-to-streamline-subsidy-voucher-program, consulté le 7 novembre 2026.

[28] Ethics and Anti-Corruption Commission (Kenya). « EACC to monitor compliance of the National Fertilizer Subsidy Programme. » [en ligne], https://eacc.go.ke/en/default/eacc-to-monitor-compliance-of-national-fertilizer-subsidy-program-in-the-ministry-of-agriculture/, consulté le 7 novembre 2026.

[29] Institute of Economic Affairs (Kenya). « The pros and cons of the KSh 3 500 fertilizer subsidy. » [en ligne], https://ieakenya.or.ke/blog/the-pros-and-cons-of-the-ksh-3500-fertilizer-subsidy/, consulté le 7 novembre 2026.

[30] Tanzania Fertilizer Regulatory Authority. « TFRA — Portail officiel de régulation des engrais. » [en ligne], https://www.tfra.go.tz/, consulté le 7 novembre 2026.

[31] ZATRA. « Register a fertilizer product in Tanzania. » [en ligne], https://www.zatra.co/post/register-fertilizer-product-tanzania, consulté le 7 novembre 2026.

[32] Spring. « Spring Boot Reference Documentation (3.3.5). » [en ligne], https://docs.spring.io/spring-boot/index.html, consulté le 7 novembre 2026.

[33] PostgreSQL Global Development Group. « PostgreSQL 16 Documentation. » [en ligne], https://www.postgresql.org/docs/16/index.html, consulté le 7 novembre 2026.

[34] Testcontainers. « Testcontainers for Java — Documentation. » [en ligne], https://java.testcontainers.org/, consulté le 7 novembre 2026.

[35] Redgate. « Flyway Documentation. » [en ligne], https://documentation.red-gate.com/flyway, consulté le 7 novembre 2026.

[36] OpenAPI Initiative. « OpenAPI Specification 3.0.3. » [en ligne], https://spec.openapis.org/oas/v3.0.3, consulté le 7 novembre 2026.

[37] Project Lombok. « Project Lombok — Documentation. » [en ligne], https://projectlombok.org/, consulté le 7 novembre 2026.

[38] GitHub. « GitHub Actions Documentation. » [en ligne], https://docs.github.com/en/actions, consulté le 7 novembre 2026.

[39] Docker Inc. « Docker Engine Documentation. » [en ligne], https://docs.docker.com/engine/, consulté le 7 novembre 2026.

[40] Railway. « Railway — Documentation de déploiement. » [en ligne], https://docs.railway.com/, consulté le 7 novembre 2026.

Annexes

Annexe A : Extraits de code

A.1 StockService.java — Méthode sortirFifo()

```
// Distribution FIFO par date de réception FOMI (tri croissant)
    @Transactional
    public List<LotSortie> sortirFifo(Depot depot, EngraisType engraisType, int quantite, Long
lotIdForce) {
      List<StockLot> lots;

        if (lotIdForce != null) {
        StockLot forced = stockLotRepository.findByDepotIdAndLotId(depot.getId(),
lotIdForce)
                .orElseThrow(() -> new BusinessException("Lot " + lotIdForce + " absent de ce
dépôt"));
            if (!forced.getLot().getEngraisType().getId().equals(engraisType.getId())) {
           throw new BusinessException("Le lot " + forced.getLot().getNumeroLot() + " n'est
pas de type " + engraisType.getNom());
            }
```

```
            if (forced.getQuantite() < quantite) {
            throw new BusinessException("Quantité insuffisante pour le lot " +
forced.getLot().getNumeroLot());
            }
        lots = List.of(forced);
        } else {
        lots = stockLotRepository
.findByDepotIdAndLotEngraisTypeIdAndQuantiteGreaterThanOrderByLotDateReceptionFomiAsc(
                    depot.getId(), engraisType.getId(), 0);
        lots = lots.stream()
                    .filter(sl -> sl.getLot().getDatePeremption() == null ||
!sl.getLot().getDatePeremption().isBefore(LocalDate.now()))
                    .collect(Collectors.toList());
        }

        if (lots.isEmpty()) {
        throw new BusinessException("Stock insuffisant au dépôt " + depot.getNom()
              + " (" + engraisType.getNom() + ") : aucun lot disponible (tous périmés)");
        }

      int restant = quantite;
      List<LotSortie> sorties = new ArrayList<>();
         for (StockLot stockLot : lots) {
             if (restant <= 0) {
           break;
             }
         int pris = Math.min(stockLot.getQuantite(), restant);
         stockLot.setQuantite(stockLot.getQuantite() - pris);
         restant -= pris;
         sorties.add(new LotSortie(stockLot.getLot(), pris));
         }
         if (restant > 0) {
         throw new BusinessException("Stock insuffisant au dépôt " + depot.getNom()
               + " (" + engraisType.getNom() + ") : manque " + restant + " sacs");
         }

      Stock stock = getOrCreate(depot, engraisType);
      stock.setQuantiteDisponible(stock.getQuantiteDisponible() - quantite);
      stockRepository.save(stock);
         return sorties;
    }
```

A.2 DistributionService.java — Classe et méthode distribuer()

// Service de distribution : FIFO, mouvements, distributions et audit
public class DistributionService {

```
    private final TicketRepository ticketRepository;
    private final DepotRepository depotRepository;
    private final MouvementRepository mouvementRepository;
    private final DistributionRepository distributionRepository;
    private final StockService stockService;
    private final AlerteService alerteService;
```

```
    private final AuditLogService auditLogService;
    private final EmailNotificationService emailNotificationService;

    @Transactional
    public List<DistributionResponse> distribuer(DistributionRequest req, Utilisateur agent) {
      Ticket ticket = ticketRepository.findByIdForUpdate(req.ticketId())
                .orElseThrow(() -> new ResourceNotFoundException("Ticket introuvable : " +
req.ticketId()));
      Depot depot = depotRepository.findById(req.depotId())
                .orElseThrow(() -> new ResourceNotFoundException("Dépôt introuvable : " +
req.depotId()));

        verifierPerimetreAgent(agent, depot);

        if (!List.of(StatutTicket.ACTIF, StatutTicket.PAYE, StatutTicket.EN_ATTENTE,
StatutTicket.PARTIEL)
                .contains(ticket.getStatut())) {
          throw new BusinessException("Distribution impossible : statut du ticket = " +
ticket.getStatut());
        }
      int restant = ticket.getQuantiteCommandee() - ticket.getQuantiteServie();
        if (restant < req.quantite()) {
          throw new BusinessException("Quantité demandée (" + req.quantite() + ") supérieure
au restant du ticket ("
              + restant + ")");
        }

      List<LotSortie> sorties = stockService.sortirFifo(depot, ticket.getEngraisType(),
req.quantite(), req.lotId());

      List<DistributionResponse> reponses = new ArrayList<>();
        for (LotSortie sortie : sorties) {
        Mouvement mouvement = Mouvement.builder()
                  .type(TypeMouvement.DISTRIBUTION)
                  .lot(sortie.lot())
                  .engraisType(ticket.getEngraisType())
                  .depotSource(depot)
                  .depotDest(null)
                  .quantite(sortie.quantite())
                  .agent(agent)
                  .ticket(ticket)
                  .commentaire(req.commentaire())
                  .dateMouvement(LocalDateTime.now())
                  .build();
        mouvementRepository.save(mouvement);

        Distribution distribution = distributionRepository.save(Distribution.builder()
                  .ticket(ticket)
                  .lot(sortie.lot())
                  .depot(depot)
                  .quantite(sortie.quantite())
                  .agent(agent)
                  .dateDistribution(LocalDateTime.now())
                  .referenceMouvement(mouvement)
                  .build());
```

```
            reponses.add(new DistributionResponse(distribution.getId(), ticket.getNumeroTicket(),
                    sortie.lot().getNumeroLot(), depot.getCode(), sortie.quantite(),
agent.getUsername(),
                    distribution.getDateDistribution()));
        }

        ticket.setQuantiteServie(ticket.getQuantiteServie() + req.quantite());
        ticket.mettreAJourStatut();
        ticketRepository.save(ticket);

        alerteService.verifierDepot(depot, ticket.getEngraisType());
        auditLogService.log("DISTRIBUTION_CREATE", "Ticket", ticket.getId(),
                "{\"depot\":\"" + depot.getCode() + "\",\"quantite\":" + req.quantite() + ",\"reste\":"
                        + ticket.getQuantiteRestante() + "}");
        String precision = "Distribution de " + req.quantite() + " kg effectuée au dépôt "
                + depot.getCode() + ". "
                + (ticket.getStatut() == StatutTicket.SERVI
                        ? "Votre ticket est entièrement servi."
                        : "Il vous reste " + ticket.getQuantiteRestante() + " kg à retirer.");
        emailNotificationService.notifierTicket(ticket, "DISTRIBUTION", precision);
        return reponses;
    }

    private void verifierPerimetreAgent(Utilisateur agent, Depot depot) {
        if (agent.getRole() == RoleUtilisateur.AGENT_COLLINE) {
            if (agent.getDepot() == null || !agent.getDepot().getId().equals(depot.getId())) {
                throw new BusinessException("Un agent colline ne peut distribuer que sur son
propre dépôt ("
                        + agent.getNomComplet() + ")");
            }
        }
        if (agent.getRole() == RoleUtilisateur.AGENT_ZONE) {
            if (agent.getZone() == null || !dansZone(depot, agent.getZone().getId())) {
                throw new BusinessException("Un agent zone ne peut distribuer que dans sa zone ("
                        + agent.getNomComplet() + ")");
            }
        }
    }

    private boolean dansZone(Depot depot, Long zoneId) {
        if (depot.getZone() != null && zoneId.equals(depot.getZone().getId())) {
            return true;
        }
        return depot.getColline() != null && depot.getColline().getZone() != null
                && zoneId.equals(depot.getColline().getZone().getId());
    }
}
```

Annexe B : Liste des endpoints API

Méthode	Endpoint	Description	Rôle

Méthode	Endpoint	Description	Rôle
GET	/api/agriculteurs	Liste paginée des agriculteurs	AUTH
GET	/api/agriculteurs/{id}	Détail d'un agriculteur	AUTH
GET	/api/agriculteurs/recherche	Recherche d'agriculteurs par critères	AUTH
POST	/api/agriculteurs	Créer un agriculteur	ADMIN, AGENT_ZONE, AGENT_COLLINE
PUT	/api/agriculteurs/{id}	Modifier un agriculteur	ADMIN, AGENT_ZONE, AGENT_COLLINE
GET	/api/alertes	Liste des alertes	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE
GET	/api/alertes/depot/{depotId}	Alertes d'un dépôt	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE
PATCH	/api/alertes/{id}/traiter	Marquer une alerte comme traitée	ADMIN
GET	/api/audit-logs	Consulter la piste d'audit (pagination max 200)	ADMIN
POST	/api/auth/login	Authentification et génération du JWT	PUBLIC
GET	/api/depots	Liste des dépôts	AUTH
GET	/api/depots/{id}	Détail d'un dépôt	AUTH
POST	/api/depots	Créer un dépôt	ADMIN
PUT	/api/depots/{id}	Modifier un dépôt	ADMIN
GET	/api/depots/{id}/stock	Stocks d'un dépôt	AUTH

Méthode	Endpoint	Description	Rôle
POST	/api/distributions	Distribuer des engrais (FIFO)	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE
GET	/api/distributions/ticket/{ticketId}	Distributions d'un ticket	AUTH
GET	/api/engrais-types	Liste des types d'engrais	AUTH
GET	/api/engrais-types/{id}	Détail d'un type d'engrais	AUTH
POST	/api/engrais-types	Créer un type d'engrais	ADMIN
PUT	/api/engrais-types/{id}	Modifier un type d'engrais	ADMIN
DELETE	/api/engrais-types/{id}	Supprimer un type d'engrais	ADMIN
GET	/api/export/tickets.csv	Export CSV des tickets	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_LOGISTIQUE
GET	/api/export/mouvements.csv	Export CSV des mouvements	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_LOGISTIQUE
GET	/api/provinces	Liste des provinces	AUTH
GET	/api/zones	Liste des zones	AUTH
GET	/api/provinces/{id}/zones	Zones d'une province	AUTH
GET	/api/collines	Liste des collines	AUTH
GET	/api/zones/{id}/collines	Collines d'une zone	AUTH
GET	/api/lots	Liste des lots	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE, AGENT_LOGISTIQUE

Méthode	Endpoint	Description	Rôle
GET	/api/lots/recherche	Recherche d'un lot par numéro	ADMIN, SUPERVISEUR, AGENT_ZONE, AGENT_COLLINE, AGENT_LOGISTIQUE
POST	/api/lots	Créer un lot	ADMIN, AGENT_LOGISTIQUE
POST	/api/mouvements/reception	Réception FOMI	ADMIN, AGENT_LOGISTIQUE
POST	/api/mouvements/transfert	Transfert inter-dépôts	ADMIN
POST	/api/mouvements/{id}/annuler	Annuler un mouvement	AUTH (règle ADMIN/SUPERVISEUR non appliquée)
GET	/api/mouvements	Liste paginée des mouvements	AUTH
GET	/api/mouvements/lot/{lotId}	Mouvements d'un lot	AUTH
GET	/api/public/tickets/{numero}	Suivi public d'un ticket	PUBLIC
GET	/api/seuils-alerte	Liste des seuils d'alerte	AUTH
POST	/api/seuils-alerte	Créer un seuil d'alerte	ADMIN
PUT	/api/seuils-alerte/{id}	Modifier un seuil d'alerte	ADMIN
DELETE	/api/seuils-alerte/{id}	Supprimer un seuil d'alerte	ADMIN
GET	/api/statistiques/taux-service	Taux de service	ADMIN, SUPERVISEUR
GET	/api/statistiques/stocks-par-province	Stocks par province	ADMIN, SUPERVISEUR
GET	/api/statistiques/ruptures	Ruptures en cours	ADMIN, SUPERVISEUR
GET	/api/statistiques/tickets-non-servis-par-colline	Tickets non servis par colline	ADMIN, SUPERVISEUR

Méthode	Endpoint	Description	Rôle
GET	/api/statistiques/taux-service-par-province	Taux de service par province	ADMIN, SUPERVISEUR
GET	/api/statistiques/mouvements-par-agent/{agentId}	Mouvements d'un agent	ADMIN, SUPERVISEUR
GET	/api/stocks	Liste des stocks	AUTH
GET	/api/stocks/disponibles	Stocks disponibles pour un type d'engrais	AUTH
GET	/api/tickets	Liste paginée des tickets	AUTH
GET	/api/tickets/{id}	Détail d'un ticket	AUTH
POST	/api/tickets	Créer un ticket	ADMIN, AGENT_ZONE, AGENT_COLLINE
PUT	/api/tickets/{id}	Modifier un ticket	ADMIN, AGENT_ZONE
PATCH	/api/tickets/{id}/activer	Activer un ticket	ADMIN, SUPERVISEUR
PATCH	/api/tickets/{id}/rejeter	Rejeter un ticket	ADMIN, SUPERVISEUR
GET	/api/utilisateurs	Liste des utilisateurs	ADMIN
GET	/api/utilisateurs/{id}	Détail d'un utilisateur	ADMIN
POST	/api/utilisateurs	Créer un utilisateur	ADMIN
PUT	/api/utilisateurs/{id}	Modifier un utilisateur	ADMIN

Annexe C : Schéma de la base de données (extrait migration SQL)

Extrait consolidé des migrations V1, V7 et V11 :

```
CREATE TYPE niveau_depot AS ENUM ('PROVINCE', 'ZONE', 'COLLINE');
CREATE TYPE statut_ticket AS ENUM ('PAYE', 'EN_ATTENTE', 'PARTIEL',
  'SERVI', 'RECLAMATION', 'ANNULE');

ALTER TYPE statut_ticket ADD VALUE 'EN_ATTENTE_DE_VALIDATION' AFTER 'EN_ATTENTE';
```

```
ALTER TYPE statut_ticket ADD VALUE 'ACTIF' AFTER 'EN_ATTENTE_DE_VALIDATION';
ALTER TYPE statut_ticket ADD VALUE 'REJETE';
ALTER TYPE statut_ticket ADD VALUE 'EXPIRE';
CREATE TYPE role_utilisateur AS ENUM ('ADMIN', 'AGENT_ZONE',
  'AGENT_COLLINE', 'SUPERVISEUR');

ALTER TYPE role_utilisateur ADD VALUE 'AGENT_LOGISTIQUE';
CREATE TYPE type_mouvement AS ENUM ('RECEPTION_FOMI', 'TRANSFERT',
  'DISTRIBUTION', 'AJUSTEMENT');

CREATE TYPE type_alerte AS ENUM ('STOCK_BAS', 'RUPTURE',
  'DELAI_DEPASSE', 'DEMANDE_SUP_STOCK');

ALTER TYPE type_alerte ADD VALUE 'PEREMPTION';
ALTER TYPE type_alerte ADD VALUE 'INACTIVITE';
CREATE TYPE niveau_alerte AS ENUM ('INFO', 'WARNING', 'CRITICAL');


CREATE TABLE depot (
  id            BIGSERIAL PRIMARY KEY,
  code          VARCHAR(30)   NOT NULL UNIQUE,
  nom           VARCHAR(150)  NOT NULL,
  niveau        niveau_depot  NOT NULL,
  province_id   BIGINT REFERENCES province(id),
  zone_id       BIGINT REFERENCES zone(id),
  colline_id    BIGINT REFERENCES colline(id),
  seuil_alerte_min INTEGER NOT NULL DEFAULT 100,
  capacite_max  INTEGER,
  actif         BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMP NOT NULL DEFAULT NOW()
);


CREATE TABLE stock (
  id            BIGSERIAL PRIMARY KEY,
  depot_id      BIGINT NOT NULL REFERENCES depot(id),
  engrais_type_id  BIGINT NOT NULL REFERENCES engrais_type(id),
```

```
    quantite_disponible INTEGER NOT NULL DEFAULT 0 CHECK (quantite_disponible >=
0),
    updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE (depot_id, engrais_type_id)
);


CREATE TABLE ticket (
    id              BIGSERIAL PRIMARY KEY,
    numero_ticket       VARCHAR(50) NOT NULL UNIQUE,
    agriculteur_id      BIGINT NOT NULL REFERENCES agriculteur(id),
    engrais_type_id     BIGINT NOT NULL REFERENCES engrais_type(id),
    quantite_commandee INTEGER NOT NULL CHECK (quantite_commandee > 0),
    quantite_servie     INTEGER NOT NULL DEFAULT 0 CHECK (quantite_servie >= 0),
    montant_paye        DECIMAL(12,2) NOT NULL,
    date_paiement       DATE NOT NULL,
    depot_retrait_id    BIGINT NOT NULL REFERENCES depot(id),
    statut              statut_ticket NOT NULL DEFAULT 'PAYE',
    saison              VARCHAR(10) NOT NULL DEFAULT '2026A',
    superficie_ha       DECIMAL(8,2),
    date_creation       TIMESTAMP NOT NULL DEFAULT NOW()
);


CREATE INDEX idx_ticket_statut ON ticket(statut);
CREATE INDEX idx_ticket_depot_retrait ON ticket(depot_retrait_id, statut);


CREATE TABLE mouvement (
    id              BIGSERIAL PRIMARY KEY,
```

```
    type            type_mouvement NOT NULL,
    lot_id          BIGINT NOT NULL REFERENCES lot(id),
    engrais_type_id  BIGINT NOT NULL REFERENCES engrais_type(id),
    depot_source_id  BIGINT REFERENCES depot(id),
    depot_dest_id    BIGINT REFERENCES depot(id),
    quantite        INTEGER NOT NULL CHECK (quantite > 0),
    agent_id        BIGINT NOT NULL REFERENCES utilisateur(id),
    ticket_id       BIGINT REFERENCES ticket(id),
    date_mouvement   TIMESTAMP NOT NULL DEFAULT NOW(),
    idempotency_key  VARCHAR(64) UNIQUE,
    annule          BOOLEAN NOT NULL DEFAULT FALSE,
    motif_annulation VARCHAR(500),
    annule_par_id    BIGINT REFERENCES utilisateur(id),
    reference_annulation_id BIGINT REFERENCES mouvement(id)
);
```

Annexe D : Résultats des tests

Résultats globaux : 218 tests exécutés, 218 réussis, 0 échoué, 0 ignoré.

Temps d'exécution total : environ 3 minutes (dont démarrage de l'instance PostgreSQL Testcontainers).

Couverture de code : non mesurée par un outil dédié.

Détail par catégorie :

- Tests d'intégration (PostgreSQL via Testcontainers) : 215 tests, 215 réussis
- Tests unitaires (DatabaseUrlConfigTest) : 3 tests, 3 réussis

- Total : 218 tests, 218 réussis